

Step Tracker Data Sheet

Table of Contents

System Overview and Specifications	3
System and Application Introduction	3
Highlighted System Features	3
Functional and Hardware Requirements List	4
System Block Diagram	5
Signal Mapping Table	5
Final Schematic Design	6
Top Level.....	6
Power Supply	7
Accelerometer Interface	8
Analog Filter.....	9
Peripherals.....	10
MCU Layout	11
Final PCB Design (ECAD).....	12
PCB Print	12
PCB 3D Print	14
Bill of Materials (BOM)	15
PCB Testing and Validation	16
PCB Subsystem Testing & Validation	16
Final Enclosure Design (MCAD).....	24
Technical Drawings of All Parts	24
Exploded View Including ECAD Parts	26
3D Models and 3D Assembly Including ECAD Parts	27
Bill of Materials (BOM)	29
Final Embedded Software.....	30
User Guide and Documentation	30
Flow Chart of Step Tracker Program & Subroutines.....	30
Embedded Software Testing and Validation.....	40
Embedded Software and Subroutine Testing & Validation	40
Test Report With All Specifications Recorded	45

System Overview and Specifications

System and Application Introduction

StepTracker is a wearable embedded step counting and walking pace identification system. The system uses a three-axis ADXL335 MEMS accelerometer to detect and count steps in real time, classifying the user's walking pace into one of three speed categories and presenting all output data on an onboard 128x64 OLED display. The system is designed for both bench-top operation via an external laboratory power supply and portable mobile operation powered by an onboard 3.7V LiPo battery, with a regulated 3.3V rail supplying all active components. User interaction is handled through a rotary encoder mounted on the PCB shield, allowing the user to navigate through the system's subroutines and initiate calibration.

Highlighted System Features

- Dual power input supporting both a 2.0mm barrel connector and a 3.7V LiPo battery via JST-2PIN connector
- Fully regulated 3.3V system rail via AP2112K LDO with input and output decoupling
- Input protection including reverse polarity MOSFET, polyfuse, and TVS transient suppression
- Three-axis MEMS accelerometer interfaced via Amphenol Minitek127 connector
- Hardware analog low pass filtering on all three accelerometer axes prior to ADC sampling
- BNC connector on filtered Z-axis output for lab measurement equipment
- Real-time step detection using peak detection on filtered X and Y axis signals
- Walking pace classification into three speed categories based on inter-step timing
- 128x64 I2C OLED display for step count and system status output
- Rotary encoder for user input and subroutine navigation
- ADXL335 self-test routine for sensor validation
- Onboard test points at key power rail nodes for validation and debugging

Functional and Hardware Requirements List

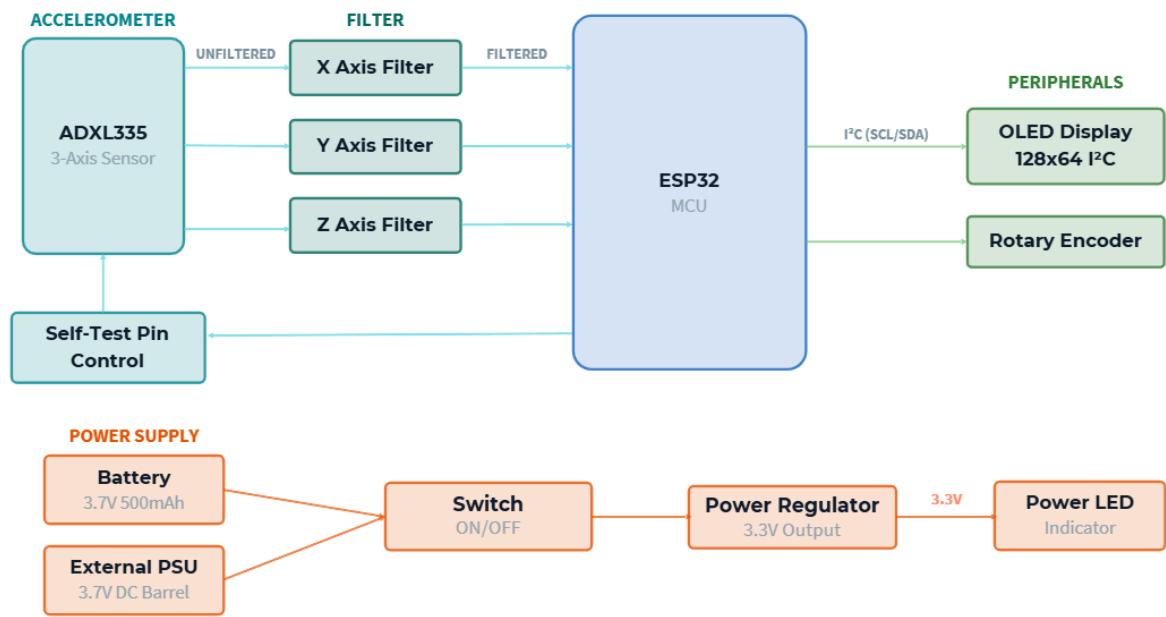
Functional Requirements Table

ID	Requirement
FR1	The system shall accept power from two sources: a 2.0mm barrel connector for external laboratory supply and a JST-2PIN connector for a 3.7V LiPo battery
FR2	The system shall produce a stable regulated 3.3V output rail to power all subsystems
FR3	The system shall be switchable on and off via a slide switch accessible to the user
FR4	Each power input shall have a dedicated indicator LED to show which source is active
FR5	The 3.3V output rail shall have a power-on indicator LED visible to the user
FR6	The system shall protect against reverse polarity, overcurrent, and transient voltage events on the input rail
FR7	The power supply shall provide sufficient current to power the ESP32, ADXL335, OLED display, indicator LEDs, and rotary encoder simultaneously

Hardware Requirements Table

ID	Requirement
HR1	The design shall conform to AS/NZS 3000 wiring standards and all UTS guidelines
HR2	The PCB shall be minimal in form factor, housing all components within the smallest reasonable area
HR3	Test points shall be placed at key nodes including the raw input rail, the post-protection rail, the LDO input, the 3.3V output rail, and ground, allowing validation of the supply at each stage of the signal path
HR4	The input protection stage shall include a polyfuse, a P-channel MOSFET for reverse polarity protection, and a TVS diode for transient suppression
HR5	The LDO regulator shall be the AP2112K-3.3TRG1 or an equivalent SOT-25 compatible device rated for a minimum of 600mA output current
HR6	Decoupling capacitors shall be placed at both the input and output of the LDO regulator to ensure stability and transient response
HR7	The power switch shall be physically accessible through the system enclosure

System Block Diagram

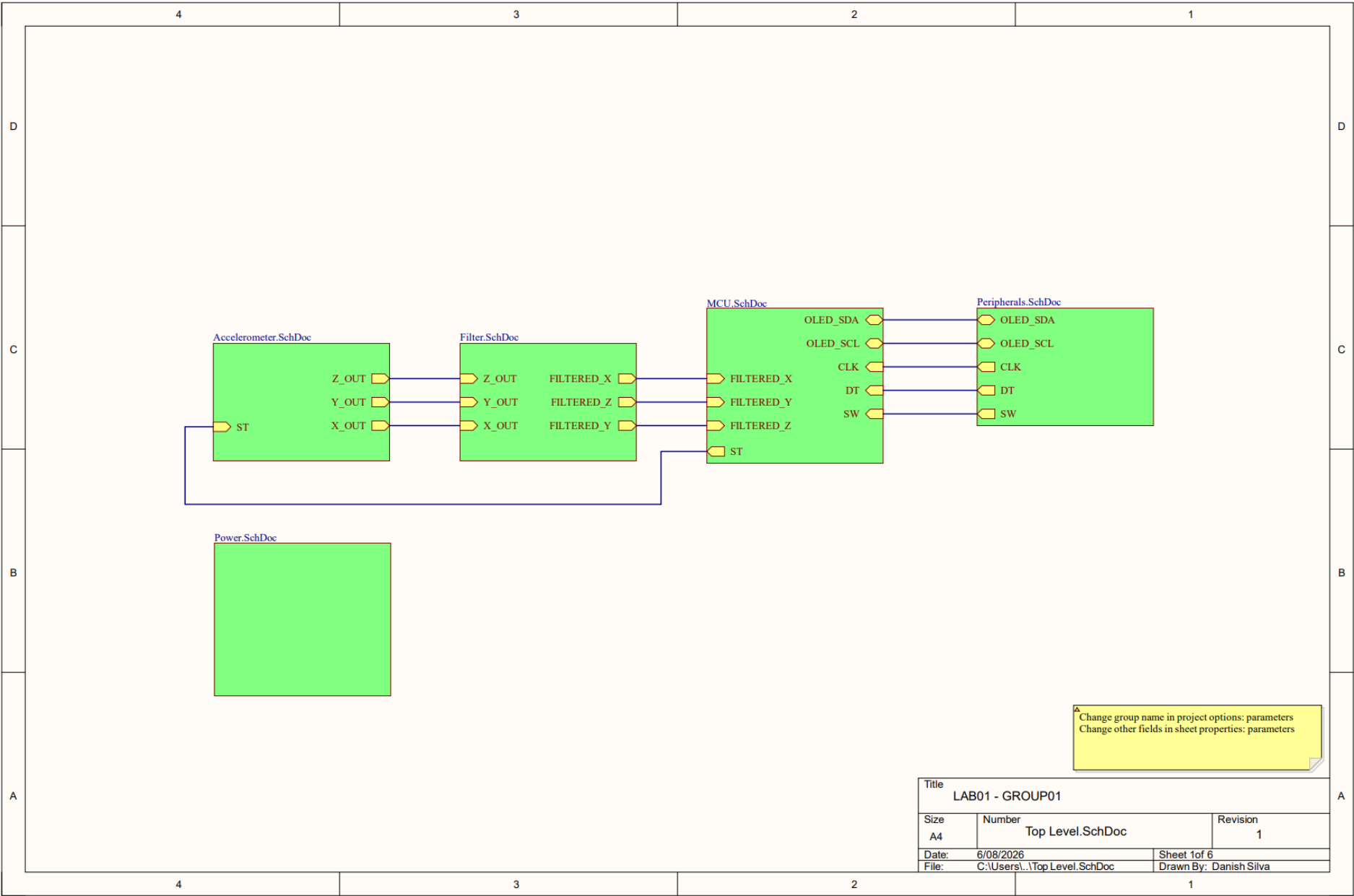


Signal Mapping Table

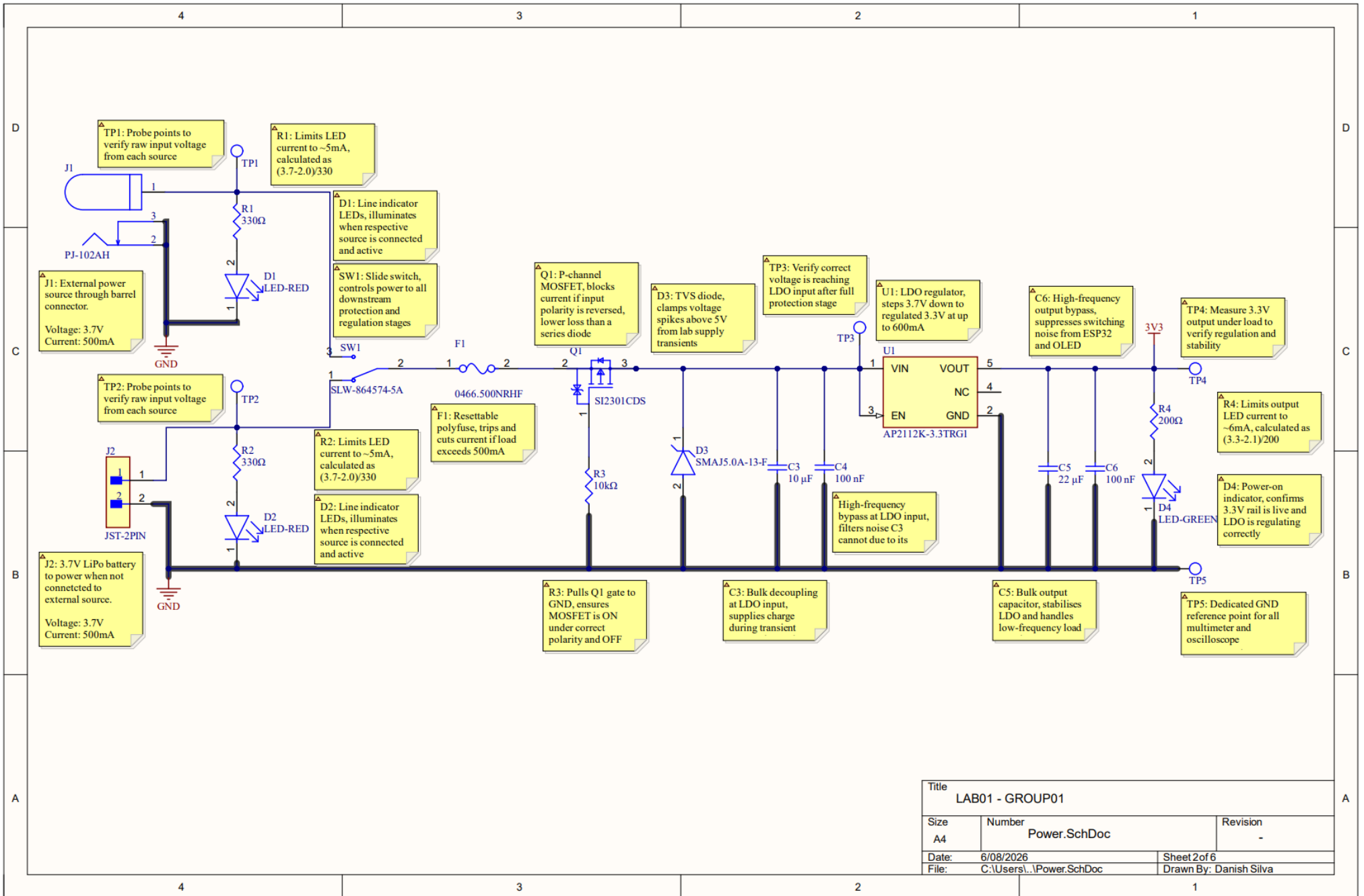
Peripheral	Signal	ESP32 GPIO	Notes
ADXL335	X Output (Filtered)	GPIO14	Via Analog Low Pass Filter
ADXL335	Y Output (Filtered)	GPIO13	Via Analog Low Pass Filter
ADXL335	Z Output (Filtered)	GPIO12	Via Analog Low Pass Filter
ADXL335	ST	GPIO33	For Self Test
OLED Display	SDA	GPIO21	I2C Clock
OLED Display	SCL	GPIO22	I2C Clock
OLED Display	I2C Address	0x3C	
Rotary Encoder	CLK	GPIO25	
Rotary Encoder	DT	GPIO36	
Rotary Encoder	SW (Button)	GPIO27	User Input/Navigation

Final Schematic Design

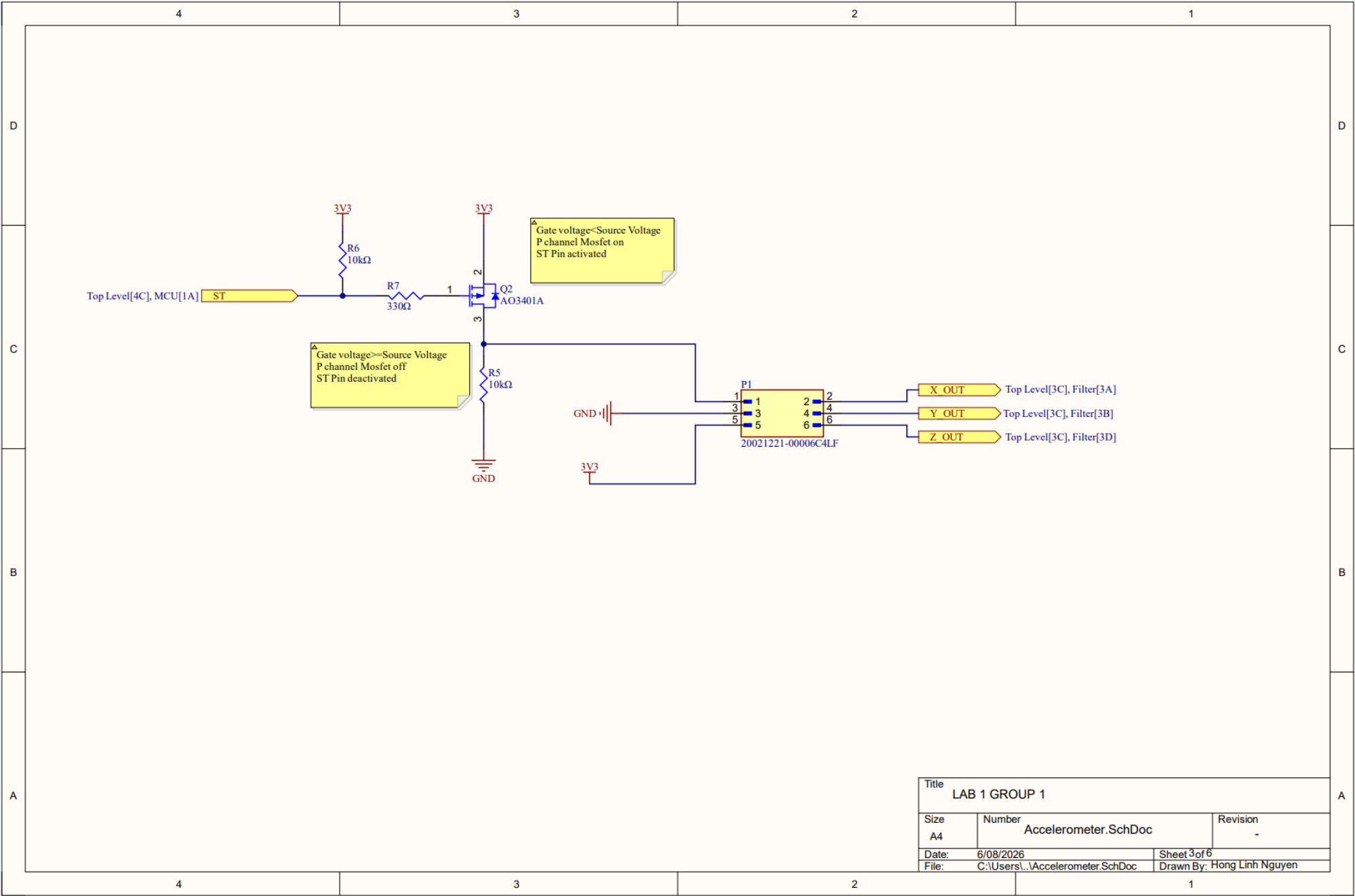
Top Level



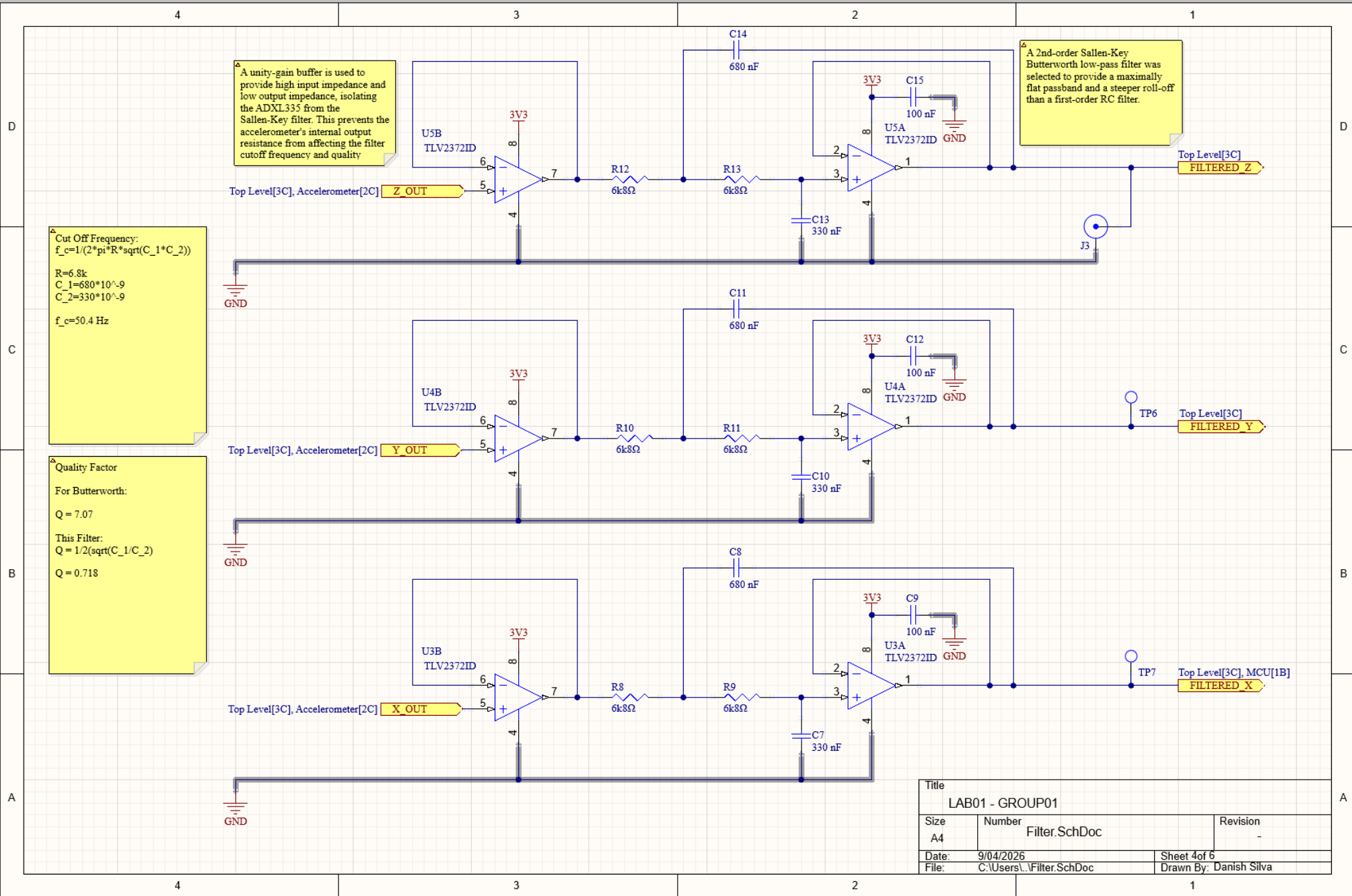
Power Supply



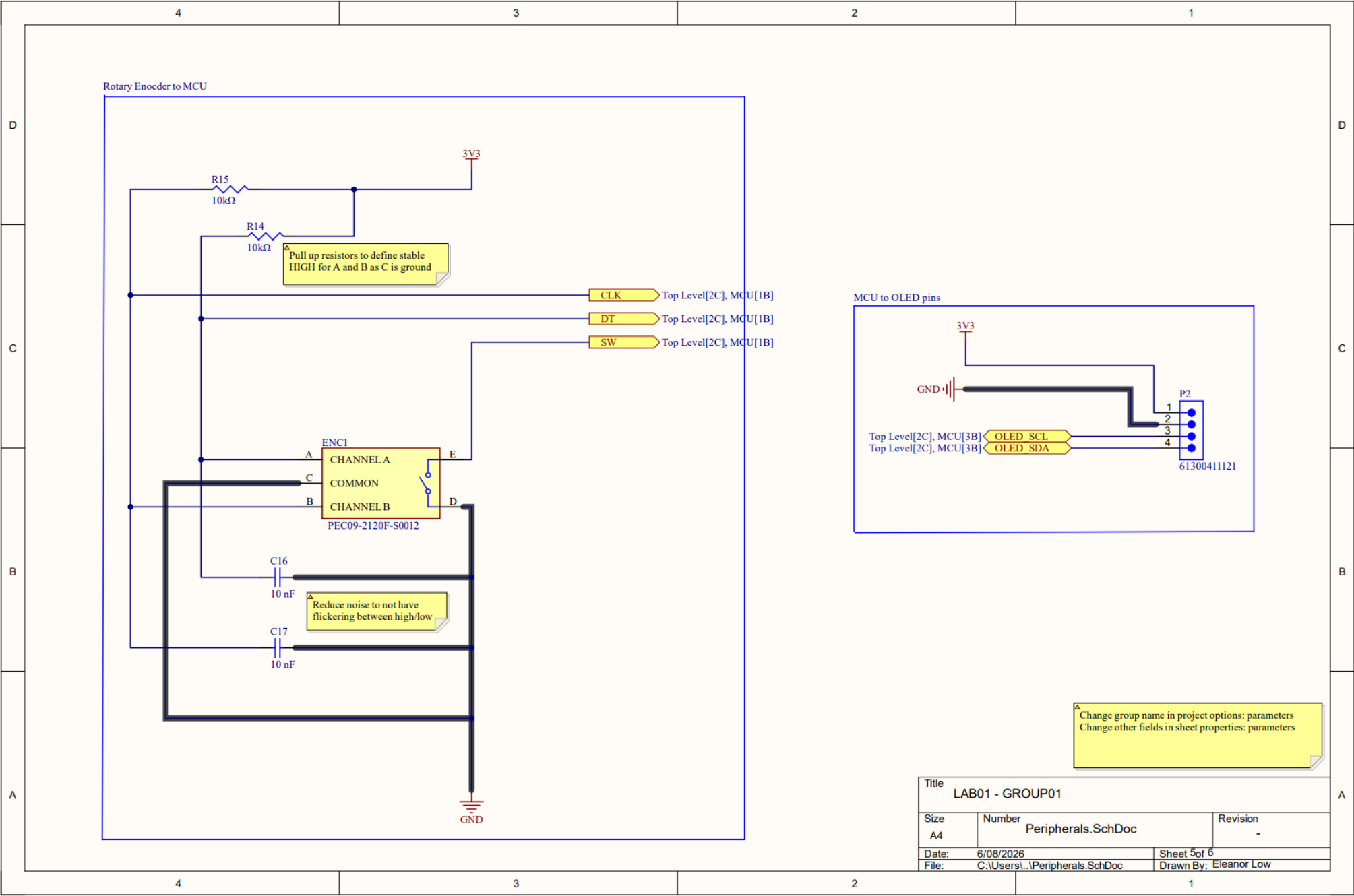
Accelerometer Interface



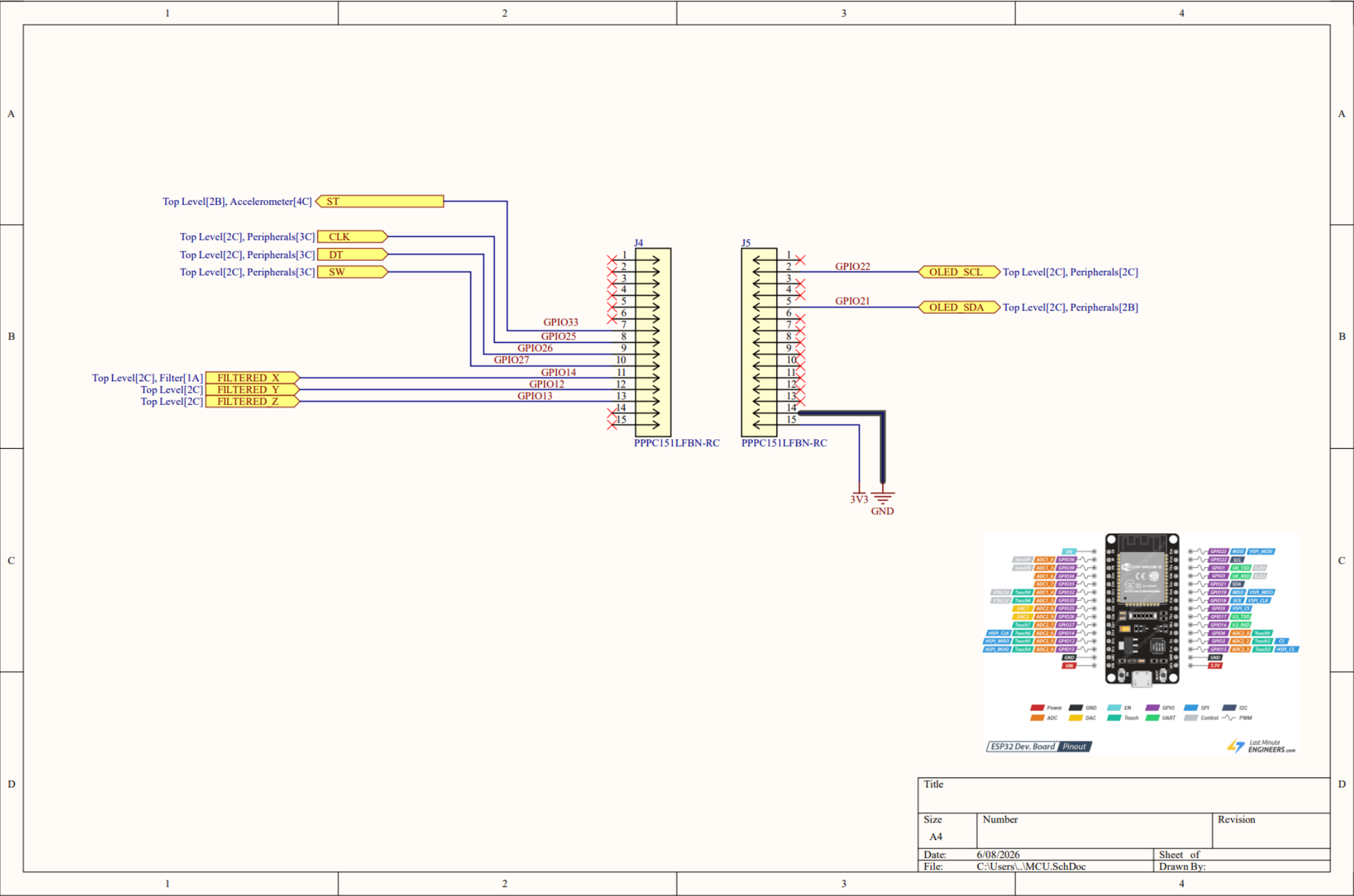
Analog Filter



Peripherals

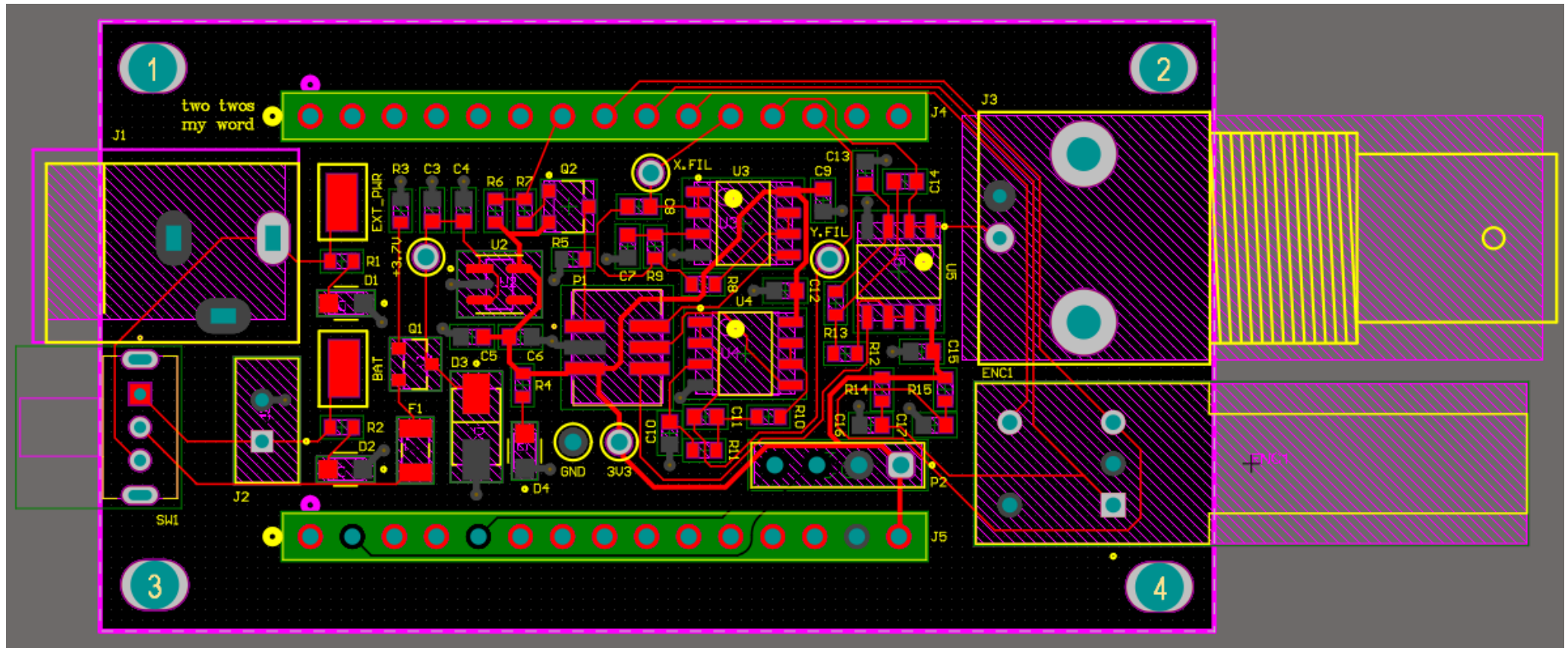


MCU Layout

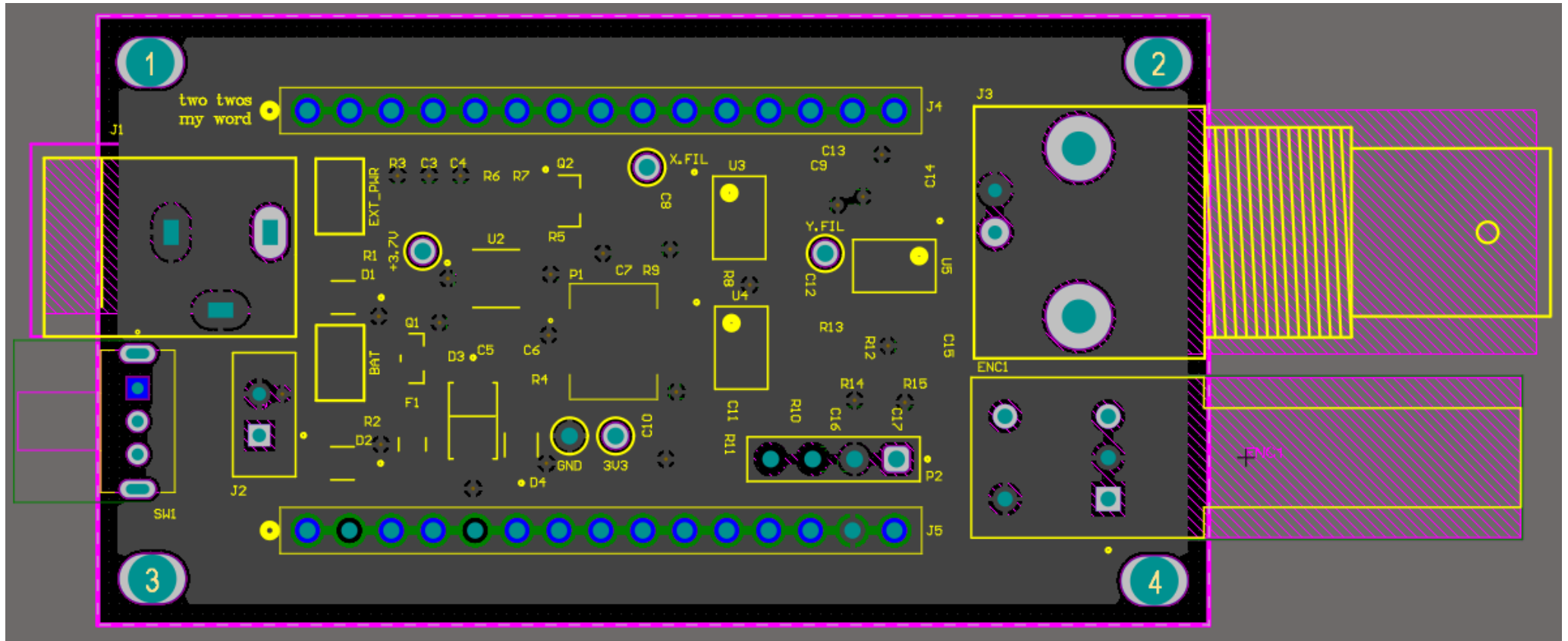


Final PCB Design (ECAD)

PCB Print



Top Layer



Bottom Layer

Bill of Materials (BOM)

No	Name	Description	Designator	Quantity	Supplier
1	PJ-102AH	Power Barrel Con, 2.0mm ID, 5.5mm OD, 5A	J1	1	DigiKey
2	B2B-EH-A(LF)(SN)	CONN HEADER VERT 2POS 2.5MM	J2	1	DigiKey
3	SLW-864574-5A	Switch	SW1	1	DigiKey
4	0466.500NRHF	FUSE BOARD MOUNT 500MA 63VAC/VDC	F1	1	DigiKey
5	SI2301CDS	MOSFET P-CH 20V 600MA SOT23-3	Q1	1	DigiKey
6	LTST-C170AKT	LED RED CLEAR SMD	D1, D2	2	DigiKey
7	LTST-C170AKT	LED GREEN CLEAR SMD	D4	1	DigiKey
8	SMAJ5.0A-13-F	TVS DIODE 5V 9.2V SMA	D3	1	DigiKey
9	AP2112K-3.3TRG1	IC REG LINEAR 3.3V 600MA SOT25	U1	1	DigiKey
10	20021221-00006C4LF	Minitex127	P1	1	DigiKey
11	AO3401A	P-Channel MOSFET, 30 V, 4 A, -55 to 150 degC, 3-Pin SOT-23-3L, RoHS, Tape and Reel	Q2	1	DigiKey
12	TLV2372ID	550 uA/Ch, 3 MHz, RRIO Operational Amplifier, 2.7 to 16 V, -40 to 125 degC, 8-pin SOIC (D8), Green (RoHS & no Sb/Br)	U3, U4, U5	3	DigiKey
13	PEC09-2120F-S0012	ROTARY ENCODER MECHANICAL 12PPR	ENC1	1	DigiKey
14	61300411121	THT Vertical Pin Header WR-PHD, Pitch 2.54 mm, Single Row, 4 pins	P2	1	DigiKey
15	1.3 Inch OLED Display Module	IIC I2C SH1106 128X64 4Pin Blue/White OLED Screen Module	N/A	1	AliExpress
16	CON BNC JACK R/A 50 OHM PCB	BNC Connector Jack, Female Socket 50Ohm Panel Mount, Through Hole, Right Angle Solder	J3	1	DigiKey
17	10 μ F	CC0603 Series	C3	1	DigiKey
18	100 nF	CC0603 Series	C4, C6	2	DigiKey
19	22 μ F	CC0603 Series	C5	1	DigiKey
20	330 nF	CC0603 Series	C7, C10, C13	3	DigiKey
21	680 nF	CC0603 Series	C8, C11, C14	3	DigiKey
22	10 nF	CC0603 Series	C16, C17	2	DigiKey
23	200 Ω	RC0603 Series 1% 1/10W	R4	1	DigiKey
24	330 Ω	RC0603 Series 1% 1/10W	R1, R2, R7	3	DigiKey
25	6k8 Ω	RC0603 Series 1% 1/10W	R8, R9, R10, R11, R12, R13	6	DigiKey
26	10k Ω	RC0603 Series 1% 1/10W	R3, R5, R6, R14, R15	5	DigiKey

PCB Testing and Validation

PRELIMINARY

No	Test	Pass / Fail	Notes
1	Perform a visual inspection of the PCB (traces, footprints, visual damage etc.).	Pass	
2	Check for Power to GND faults on all power supply rails (Vin, +5V, +3V).	Pass	
3	Polarity check for all diodes	Pass	
4	Test slider switch	Pass	

PCB Subsystem Testing & Validation

POWER SUPPLY

No	Test	Pass / Fail	Notes
1	Populate +3V linear voltage regular circuit. Pay attention to polarity of diodes and polarized capacitors	Pass	
2	Record voltage at +3V test point and compare with design specifications.	Pass	
3	Confirm indicator LED, record current draw and compare with design specifications.	Pass	
4	Confirm correct power supply voltage at power supply pins of +3V supply (opamps, microcontroller, display, ADXL335 etc.).	Pass	

PERIPHERALS

No	Test	Pass / Fail	Notes
1	Populate buttons.	Pass	
2	Confirm active high / active low button behaviour.	Pass	
3	Confirm rotary encoder works on scroll	Pass	
4	Confirm OLED is powered and able to receive data	Pass	

ANALOG LOW-PASS FILTERS

Frequency	Analytical Value	Simulated Value
Filter Cut-Off Frequency (f_c)	50Hz	48Hz
Filter Order and Topology	Unity-gain buffer + 2nd order Sallen-Key low-pass filter	
Filter Gain	1V/V	0.98V/V

X-Axis

No	Test	Pass / Fail	Notes
1	Populate analog filter circuit.	Pass	
2	Apply a sinusoidal signal in the filter passband ($f_c/10$) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Output measured at approximately 1Vpp with a phase shift of $\sim 8^\circ$. Amplitude closely matches the analytical passband gain of 1 V/V (0 dB) as expected, since at $f_c/10$ the filter introduces negligible attenuation. The small phase lag is consistent with the analytical value of $\arctan(0.14) \approx 8^\circ$. Results agree closely with simulation, confirming correct component values and filter population.
3	Apply a sinusoidal signal at the filter cut-off frequency (f_c) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Output measured at approximately 0.707Vpp, corresponding to -3dB attenuation as expected at the cutoff frequency of a 2nd order Butterworth filter. Phase shift measured at approximately -90° , consistent with the analytical response of a 2nd order filter at f_c . Results align well with simulation, validating the calculated cutoff frequency of 50.4 Hz.
4	Apply a sinusoidal signal in the filter stopband ($10f_c$) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Output heavily attenuated, measured at approximately 10mVpp (0.01 V/V, -40dB), consistent with the expected 2nd order roll off of -40dB/decade at one decade above f_c . Phase shift measured at approximately -172° , approaching the theoretical -180° limit of a 2nd order filter deep in the stopband. Results confirm the filter is effectively rejecting out-of-band signals above the cutoff frequency.
5	Record filter frequency response by sweeping the test frequency manually and compare with simulations.	Pass	Measured cutoff frequency confirmed at approximately 50 Hz, closely matching the analytical value of 50.4 Hz. Passband gain measured

	Record actual filter cut-off frequency and gain. Plot simulated and actual filter frequency response. Discuss the results.		at 0.99 V/V, within 1% of the analytical unity gain value. The measured frequency response curve closely follows the simulated Butterworth response with a -40dB/decade roll off observed above f_c . Minor deviations between measured and simulated response are attributed to component tolerances. The BNC output on the Z-axis was used to connect directly to the oscilloscope for frequency response measurements.
--	--	--	---

Y-Axis

No	Test	Pass / Fail	Notes
1	Populate analog filter circuit.	Pass	Output measured at approximately 1Vpp with a phase shift of $\sim 8^\circ$. Amplitude closely matches the analytical passband gain of 1 V/V (0 dB) as expected, since at $f_c/10$ the filter introduces negligible attenuation. The small phase lag is consistent with the analytical value of $\arctan(0.14) \approx 8^\circ$. Results agree closely with simulation, confirming correct component values and filter population.
2	Apply a sinusoidal signal in the filter passband ($f_c/10$) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Output measured at approximately 0.707Vpp, corresponding to -3dB attenuation as expected at the cutoff frequency of a 2nd order Butterworth filter. Phase shift measured at approximately -90° , consistent with the analytical response of a 2nd order filter at f_c . Results align well with simulation, validating the calculated cutoff frequency of 50.4 Hz.
3	Apply a sinusoidal signal at the filter cut-off frequency (f_c) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Output heavily attenuated, measured at approximately 10mVpp (0.01 V/V, -40dB), consistent with the expected 2nd order roll off of -40dB/decade at one decade above f_c . Phase shift measured at approximately -172° , approaching the theoretical -180° limit of a 2nd order filter deep in the stopband. Results confirm the filter is effectively rejecting out-of-band signals above the cutoff frequency.

4	Apply a sinusoidal signal in the filter stopband ($10f_c$) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Measured cutoff frequency confirmed at approximately 50 Hz, closely matching the analytical value of 50.4 Hz. Passband gain measured at 0.99 V/V, within 1% of the analytical unity gain value. The measured frequency response curve closely follows the simulated Butterworth response with a -40dB/decade roll off observed above f_c . Minor deviations between measured and simulated response are attributed to component tolerances. The BNC output on the Z-axis was used to connect directly to the oscilloscope for frequency response measurements.
5	Record filter frequency response by sweeping the test frequency manually and compare with simulations. Record actual filter cut-off frequency and gain. Plot simulated and actual filter frequency response. Discuss the results.	Pass	Output measured at approximately 1Vpp with a phase shift of $\sim 8^\circ$. Amplitude closely matches the analytical passband gain of 1 V/V (0 dB) as expected, since at $f_c/10$ the filter introduces negligible attenuation. The small phase lag is consistent with the analytical value of $\arctan(0.14) \approx 8^\circ$. Results agree closely with simulation, confirming correct component values and filter population.

Z-Axis

No	Test	Pass / Fail	Notes
1	Populate analog filter circuit.	Pass	Output measured at approximately 1Vpp with a phase shift of $\sim 8^\circ$. Amplitude closely matches the analytical passband gain of 1 V/V (0 dB) as expected, since at $f_c/10$ the filter introduces negligible attenuation. The small phase lag is consistent with the analytical value of $\arctan(0.14) \approx 8^\circ$. Results agree closely with simulation, confirming correct component values and filter population.
2	Apply a sinusoidal signal in the filter passband ($f_c/10$) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at	Pass	Output measured at approximately 0.707Vpp, corresponding to -3dB attenuation as expected at the cutoff frequency of a 2nd order Butterworth filter. Phase shift measured at approximately -90° , consistent with

	the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.		the analytical response of a 2nd order filter at f_c . Results align well with simulation, validating the calculated cutoff frequency of 50.4 Hz.
3	Apply a sinusoidal signal at the filter cut-off frequency (f_c) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Output heavily attenuated, measured at approximately 10mVpp (0.01 V/V, -40dB), consistent with the expected 2nd order rolloff of -40dB/decade at one decade above f_c . Phase shift measured at approximately -172° , approaching the theoretical -180° limit of a 2nd order filter deep in the stopband. Results confirm the filter is effectively rejecting out-of-band signals above the cutoff frequency.
4	Apply a sinusoidal signal in the filter stopband ($10f_c$) with 1Vpp amplitude and 1.5V DC offset to the filter input test point and record the signal at the output test point. Record signal amplitude and phase shift and compare with analytical / simulated filter response. Discuss the results.	Pass	Measured cutoff frequency confirmed at approximately 50 Hz, closely matching the analytical value of 50.4 Hz. Passband gain measured at 0.99 V/V, within 1% of the analytical unity gain value. The measured frequency response curve closely follows the simulated Butterworth response with a -40dB/decade roll off observed above f_c . Minor deviations between measured and simulated response are attributed to component tolerances. The BNC output on the Z-axis was used to connect directly to the oscilloscope for frequency response measurements.
5	Record filter frequency response by sweeping the test frequency manually and compare with simulations. Record actual filter cut-off frequency and gain. Plot simulated and actual filter frequency response. Discuss the results.	Pass	Output measured at approximately 1Vpp with a phase shift of $\sim 8^\circ$. Amplitude closely matches the analytical passband gain of 1 V/V (0 dB) as expected, since at $f_c/10$ the filter introduces negligible attenuation. The small phase lag is consistent with the analytical value of $\arctan(0.14) \approx 8^\circ$. Results agree closely with simulation, confirming correct component values and filter population.

ADXL335 INTERFACE

ST-Pin Driver Circuit

No	Test	Pass / Fail	Notes
1	Populate ADXL335 connector and confirm correct pinout.	Pass	Amphenol Minitex127 connector populated and pinout verified against connector datasheet and schematic.
2	Populate ST pin driver circuit.	Pass	P-channel MOSFET (AO3401A), gate pull-up resistor, and normally-open button populated correctly.
3	Without the ADXL335 breakout PCB connected, confirm correct functioning of the ST pin driver circuit (voltage at ST pin of connector is 3.3V when activated).	Pass	ST pin measured at 3.28V when MOSFET activated, consistent with 3.3V rail. MOSFET switching confirmed functional.

ADXL335 Self-Test

No	Test	Pass / Fail	Notes
1	Mount the ADXL335 breakout PCB (double confirm ADXL335 breakout PCB connector orientation is correct) and confirm mechanical clearances to other PCB components.	Pass	Breakout PCB seated correctly on Minitex127 connector, orientation verified, no mechanical conflicts with adjacent components.
2	Power on PCB and record signals from ADXL335 (stationary on lab bench, self-test deactivated) for all axes from analog filter output test points. Record 0g bias level (ratiometric) for all axes and compare with ADXL specifications and your design specifications.	Pass	Measured $X \approx 1.65V$, $Y \approx 1.65V$, $Z \approx 1.98V$. X and Y consistent with expected 0g bias of $VCC/2 = 1.65V$ at 3.3V supply. Z offset reflects +1g gravitational component ($\sim 330mV/g \times 1g = 0.33V$ above bias). Results align with datasheet ratiometric specifications.
3	Enable the Self-Test pin and record the change in output signals. Record outputs (DC voltage) for all axes and compare with ADXL specifications and your design specifications.	Fail	ST pin driver circuit verified functional (Test 6.1.3). However the GPIO pin assigned to drive the ST pin driver was unavailable for programming due to a pin conflict on the ESP32, preventing software activation of the self-test routine. The ST pin driver circuit hardware was confirmed working independently.

ADXL335 Sensor Signals

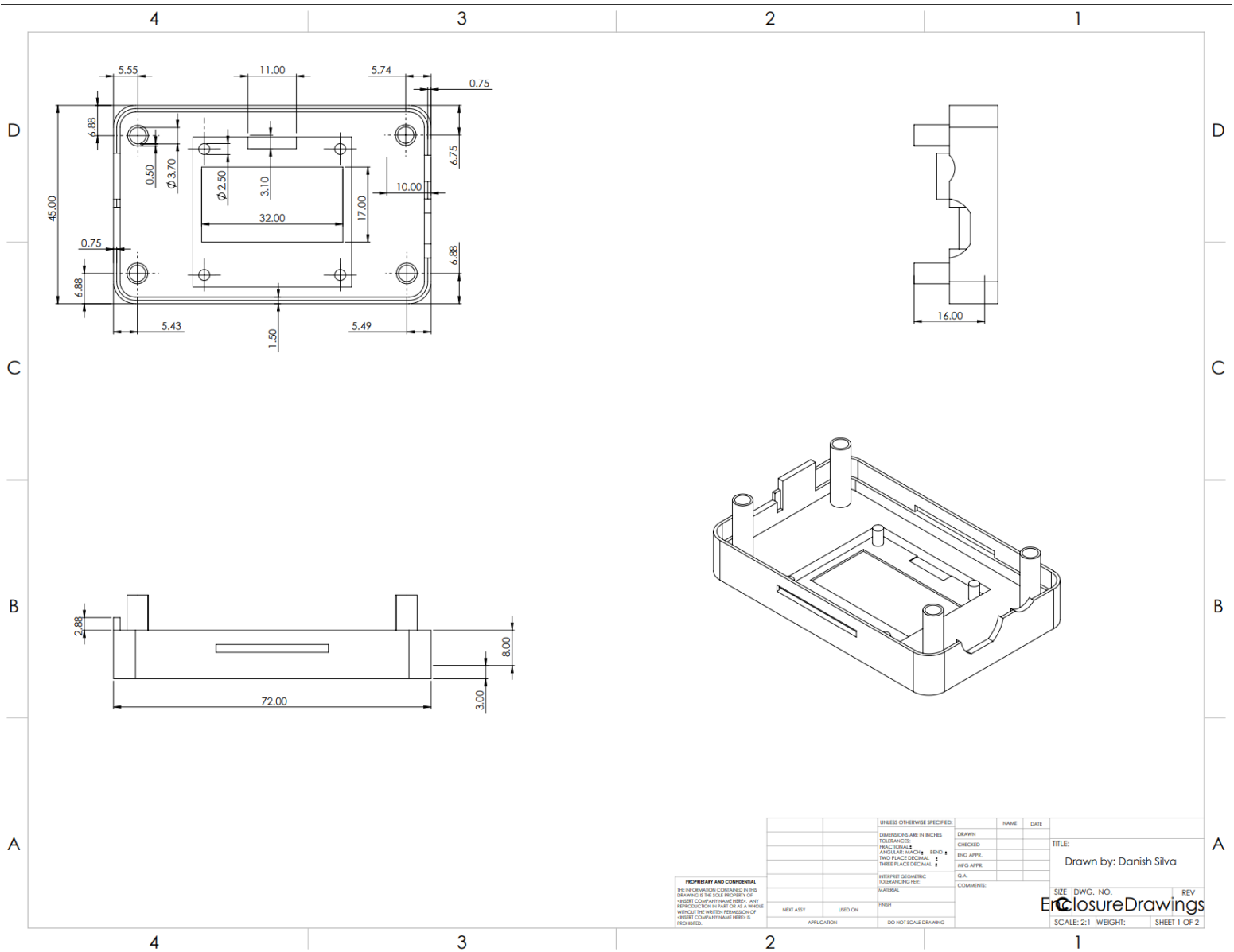
No	Test	Pass / Fail	Notes
1	Record signals from ADXL335 for all axes from filter input test points (500Hz bandwidth) when stationary. Record RMS noise and compare with ADXL335 specifications	Pass	RMS noise measured at approximately 1.1 mV (X), 1.0 mV (Y), 1.1 mV (Z) at 500Hz bandwidth. Consistent with datasheet noise density of $\sim 150 \mu\text{g}/\sqrt{\text{Hz}}$ scaled to 500Hz bandwidth and 330mV/g sensitivity at 3.3V.
2	Record signals from ADXL335 for all axes from filter output test points (low pass filter bandwidth) when stationary. Record RMS noise and compare with ADXL335 specifications.	Pass	RMS noise at filter output reduced to approximately 0.35 mV across all axes, consistent with the 50.4Hz low-pass filter bandwidth reducing noise floor as expected. Improvement confirms filter is attenuating out-of-band noise effectively.
3	Flip PCB in multiple directions and observe 1g output for respective axis. Record the sensitivity (ratiometric) for each axis and compare with datasheet and your design specifications (remember datasheet refers to 3V supply).	Pass	Sensitivity measured at approximately 330 mV/g across all axes, consistent with ratiometric scaling of datasheet value of 300 mV/g at 3V to 330 mV/g at 3.3V supply ($300 \times 3.3/3.0$). Results validated by observing expected axis output swap when PCB flipped 180°.
4	Record total current draw of PCB (without microcontroller connected).	Pass	Total current measured at approximately 19 mA, consistent with expected draw from three TLV2372ID dual op-amps ($\sim 3.3 \text{ mA}$), ADXL335 ($\sim 0.35 \text{ mA}$), indicator LEDs ($\sim 15 \text{ mA}$), and passive components.

ESP32 INTERFACE

No	Test	Pass / Fail	Notes
1	Populate pin headers for microcontroller interface, mount microcontroller and confirm mechanical clearances between PCB components and microcontroller board.	Pass	ESP32 development board mounted on PCB shield headers. Mechanical clearance confirmed between ESP32 board and all PCB components with no physical conflicts.
2	Confirm all pin header connections are correct.	Pass	All header connections verified against ESP32 pinout and schematic signal mapping. GPIO14, 13, 12 confirmed on ADC inputs, GPIO21/22 on I2C, GPIO25/36/27 on rotary encoder.
3	Power on PCB and confirm microcontroller powered on correctly (run a heart-beat LED test or similar).	Pass	ESP32 powered on successfully from 3.3V regulated rail. Heartbeat LED blink test uploaded and confirmed correct operation, validating both power delivery and USB programming interface.
4	Record total current draw of PCB with microcontroller connected.	Pass	Total current measured at approximately 115 mA with ESP32 active, consistent with expected combined draw of PCB subsystems (~19 mA) and ESP32 in active mode (~80-100 mA) with WiFi disabled.

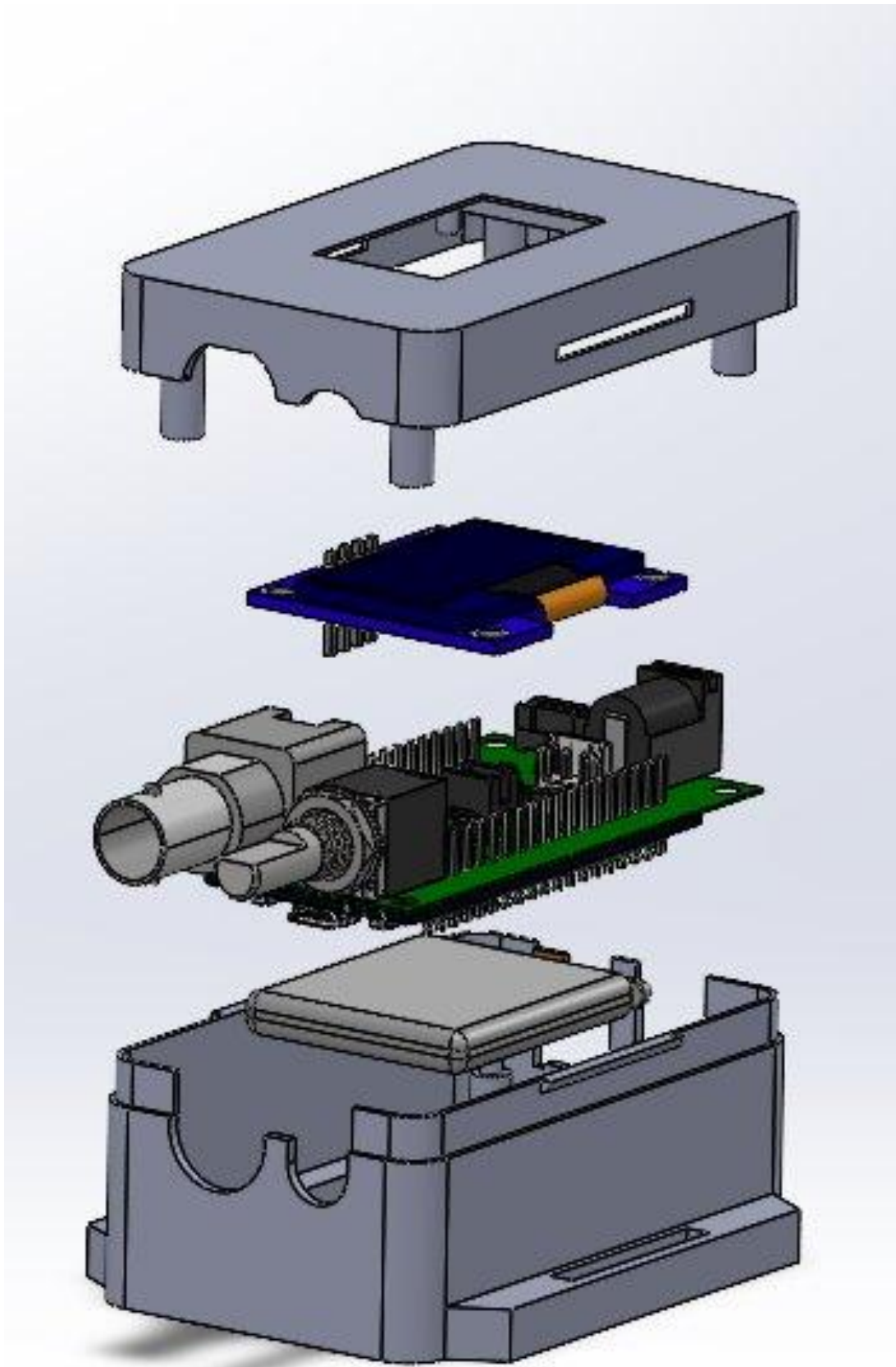
Final Enclosure Design (MCAD)

Technical Drawings of All Parts

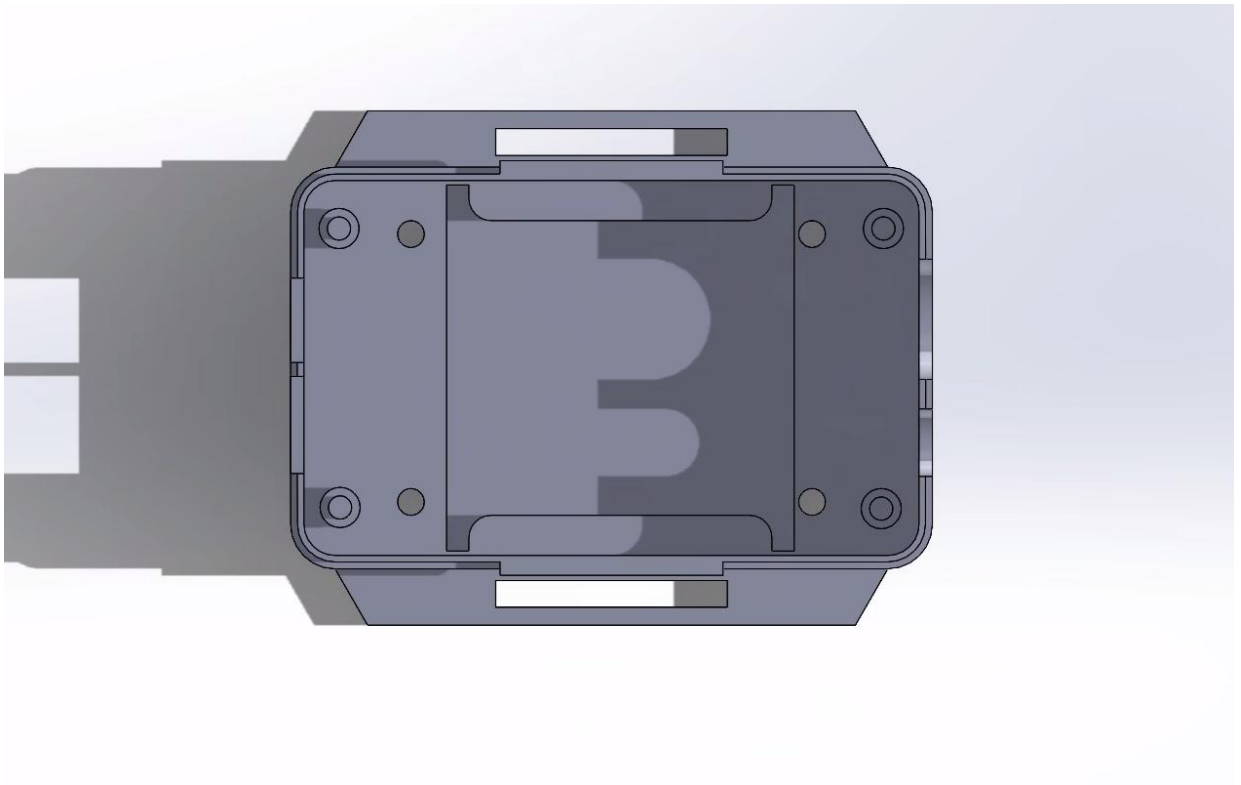
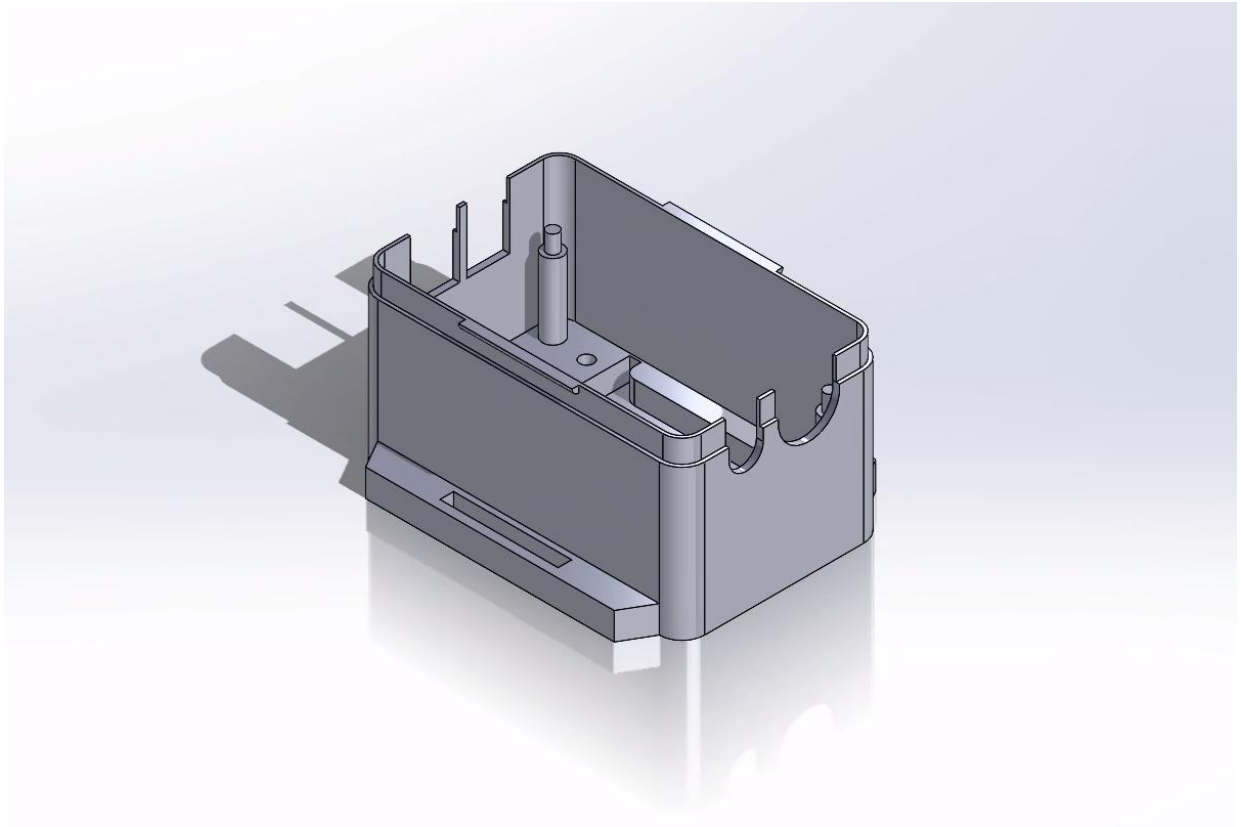


Enclosure Top Half Drawing

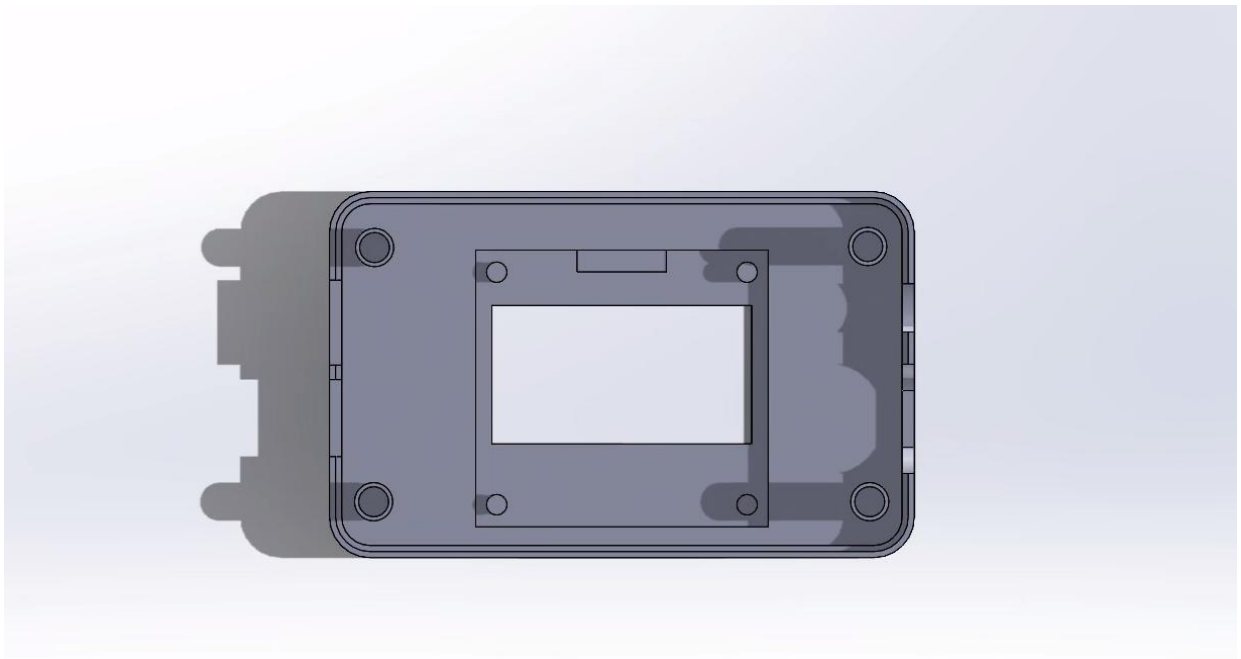
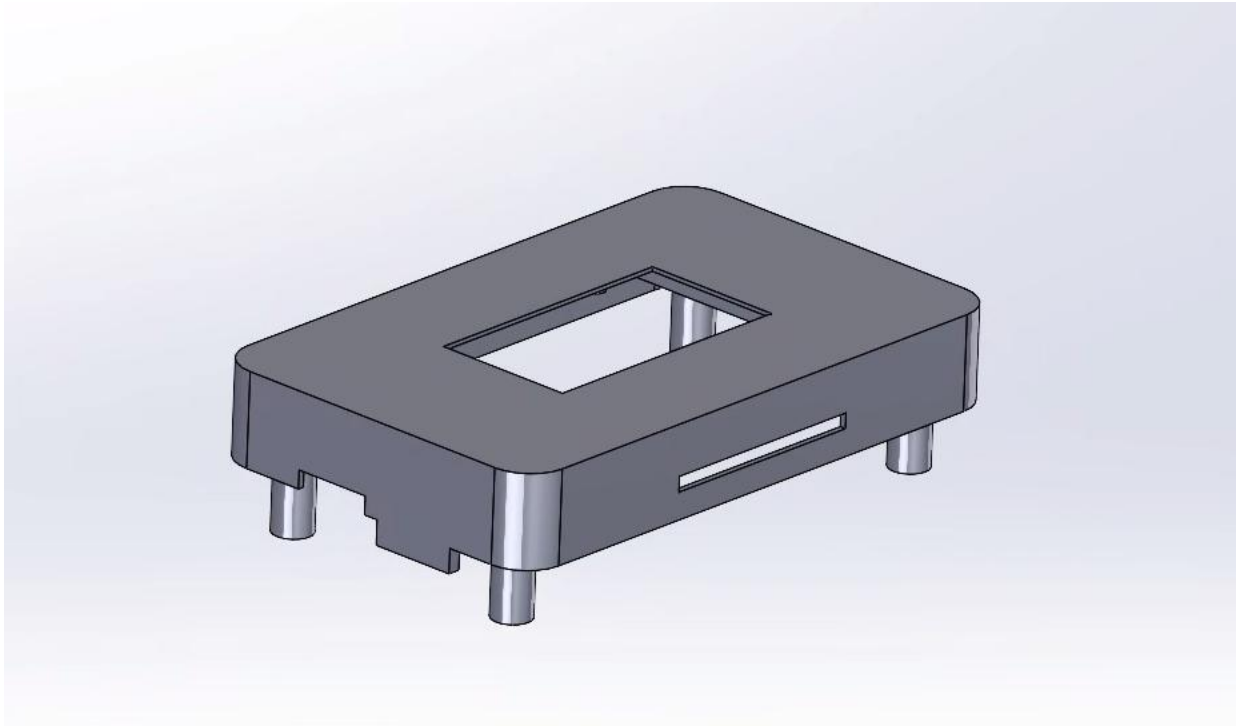
Exploded View Including ECAD Parts



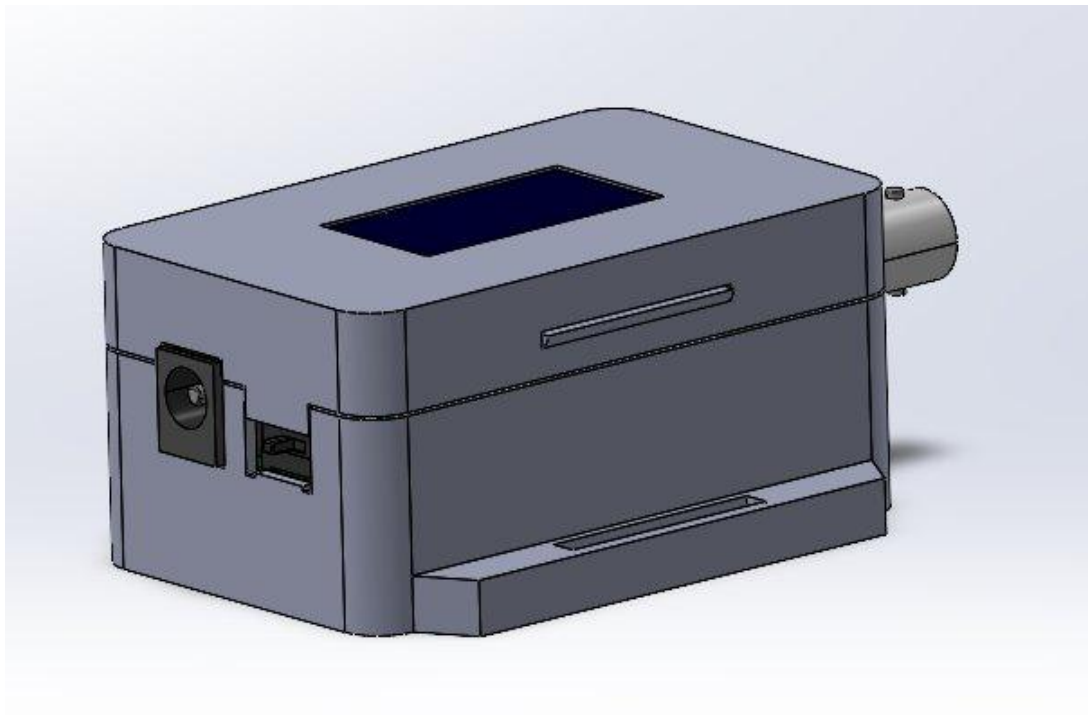
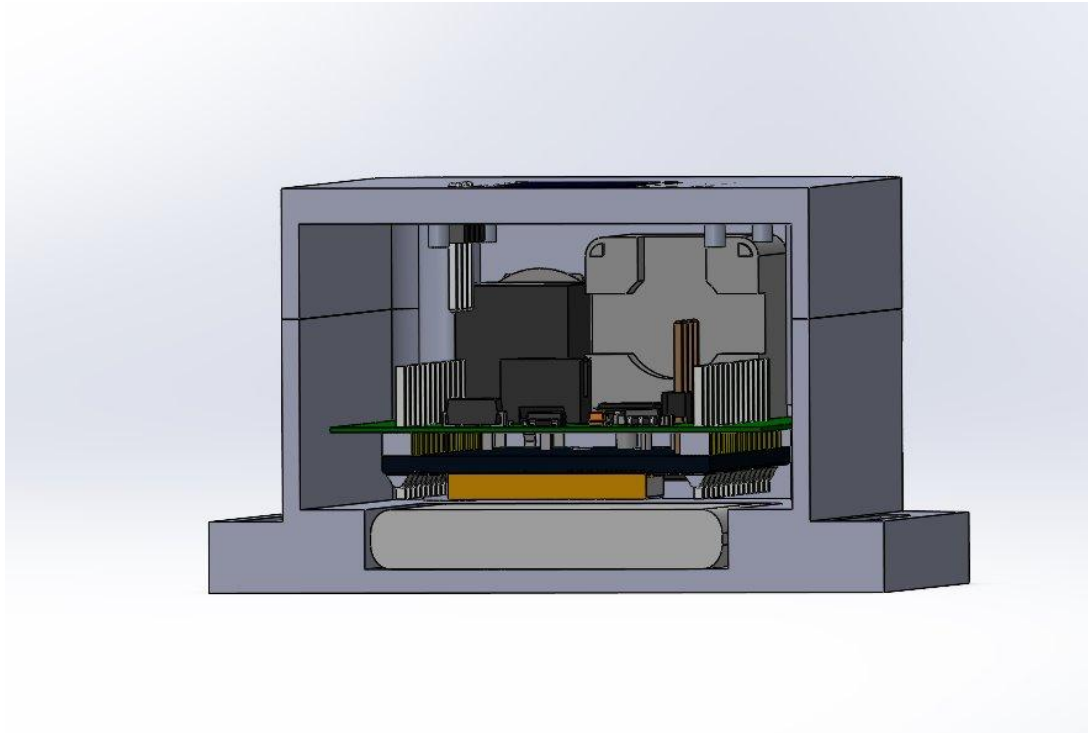
3D Models and 3D Assembly Including ECAD Parts



Enclosure Bottom Half



Enclosure Top Half



Enclosure Assembly With ECAD

Bill of Materials (BOM)

No	Name	Description	Quantity	Supplier
1	PLA	Matte Black PLA 1kg	1	Siddament
2	Wrist strap	Anko Multi – Purpose Strap – 2 pack	1	Kmart

Final Embedded Software

User Guide and Documentation

<https://github.com/DanishUTS/StepTracker>

OVERVIEW

StepTracker is an embedded step-counting device built on an ESP32 microcontroller with an ADXL335 analog accelerometer and a 1.3" SH1106 OLED display, operated through a single rotary encoder. The firmware counts walking steps in any device orientation, identifies the current walking pace (stationary / walking / running), and lets the user calibrate the accelerometer against gravity. All functions are reached through a small on-screen menu driven by the encoder.

The software is written in C++ on the Arduino framework and organised into focused modules, each responsible for one part of the system (sensor, display, input, step detection, pace, and calibration), coordinated by a central main program.

RUNNING IT

The project is built and uploaded with PlatformIO (ESP32, Arduino framework):

```
pio run          # compile
pio run -t upload # compile + flash to the board
pio device monitor # open the serial monitor at 115200 baud
```

On power-up the device shows a brief "StepTracker" splash screen for three seconds, loads any saved calibration from flash, then opens the home screen ready to count steps.

HOW TO OPERATE IT

Home Screen

The home screen is the default display and shows live step data:

STEPS	WALK	<- current pace (right)
1234		<- total steps
W:1100 R:134		<- steps while walking / running

- Pace (top-right): IDLE when stationary, WALK when walking, RUN when running.
- Total (centre): all steps counted since the last reset.
- W / R (bottom): the per-pace breakdown — walking steps and running steps (these always add up to the total).

Simply moving or walking with the device increases the count automatically; no interaction is needed for counting.

Menu Navigation

Everything is controlled with the rotary encoder:

Action	Result
Double-click the encoder button	Open the menu
Rotate the encoder	Move the highlight up/down the menu
Single-click	Select the highlighted item

The menu contains four items:

Item	What It Does
Step Counting	Returns to the home screen
Self Test	sensor self-test routine
Calibration	Starts the guided calibration routine
Reset	Sets the total and per-pace step counts back to zero and returns home

Calibration Routine

Selecting Calibration starts a guided six-step routine — one step per orientation (Z up, Z down, X up, X down, Y up, Y down). For each:

1. Place the board in the orientation shown on screen.
2. Click the encoder when it is steady.
3. Keep it still during the short "Hold still..." countdown while the device takes its measurement.

After all six orientations, the device calculates each axis's calibration, saves it to flash memory (so it survives power-off), and displays CAL DONE. Click to return home. A double-click aborts the routine at any time. Step counting is paused during calibration so that moving the board does not add false steps.

SOFTWARE ARCHTECTURE

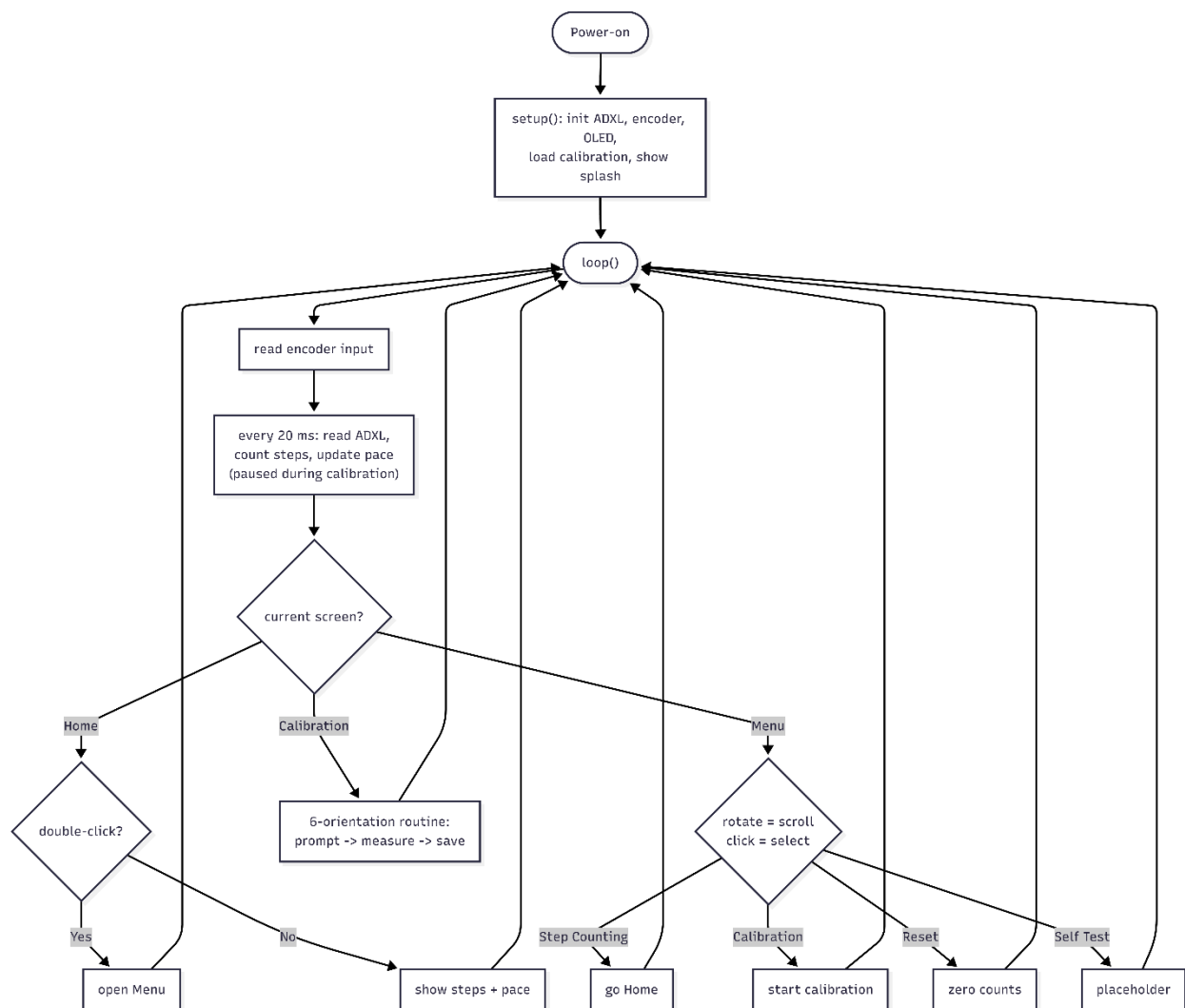
File Layout

include/	
Config.h	All pin assignments and tunable constants
src/	
main.cpp	Coordinates everything: menu state machine
Adxl335.h/.cpp	Reads the accelerometer and applies calibration
StepCounter.h/.cpp	Step-detection algorithm
PaceIdentifier.h/.cpp	Walking-pace classification and per-pace counts
Calibration.h/.cpp	Calibration data, maths, and flash storage
RotaryEncoder.h/.cpp	Encoder rotation and button handling
Display.h/.cpp	OLED screen rendering

How The Modules Work Together

main is the coordinator. On every cycle it asks RotaryEncoder for user input and, at a fixed 50 Hz rate, reads the Adxl335 accelerometer. Each reading is passed to StepCounter, which decides whether a step occurred; if so, PaceIdentifier is updated to track cadence and per-pace totals. Depending on which screen the user is on, main then calls Display to draw the home screen, the menu, or the calibration prompts. The Calibration module supplies the conversion values that Adxl335 uses, loaded from flash at start-up and updated whenever the user runs the calibration routine.

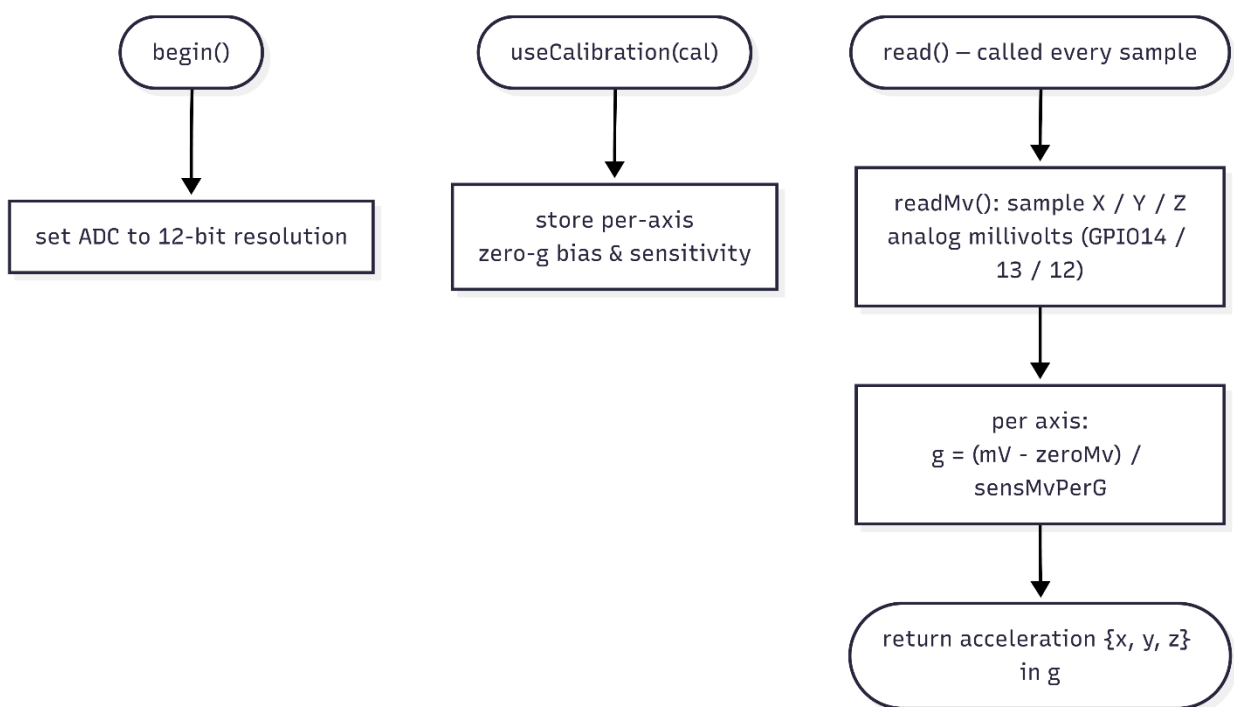
Flow Chart of Step Tracker Program & Subroutines



main.cpp — Program Orchestration & UI

- `setup()` — Runs once at power-on. Initialises the serial port, accelerometer, rotary encoder and OLED, loads the saved calibration from flash, shows the splash screen, then draws the home screen.
- `loop()` — The continuously repeating main cycle. It polls the encoder, samples the accelerometer at 50 Hz (counting steps and updating pace), then dispatches user input to the correct screen via a state machine.
- `renderHome(force)` — Draws the home screen (total steps, current pace, and walk/run breakdown), but only redraws when a value changes to avoid flooding the display.
- `openMenu()` — Switches to the menu screen and resets the highlight to the first item.
- `activate(i)` — Runs the action for the selected menu item (return home, open self-test, start calibration, or reset the counts).
- `startCalibration()` — Begins the calibration routine: resets to step 0 and the first prompt.

- `captureCalStep()` — Averages a burst of readings for the current orientation and stores the result for that axis.
- `finishCalibration()` — After all six orientations, computes each axis's calibration, saves it to flash, applies it, and prints the values.
- `renderCalibrate(now)` — Draws the calibration screen for the current phase (prompt, countdown, or done), redrawing only on change.
- `axisMv(m, axis)` — Small helper that picks one axis's value from a raw three-axis reading.

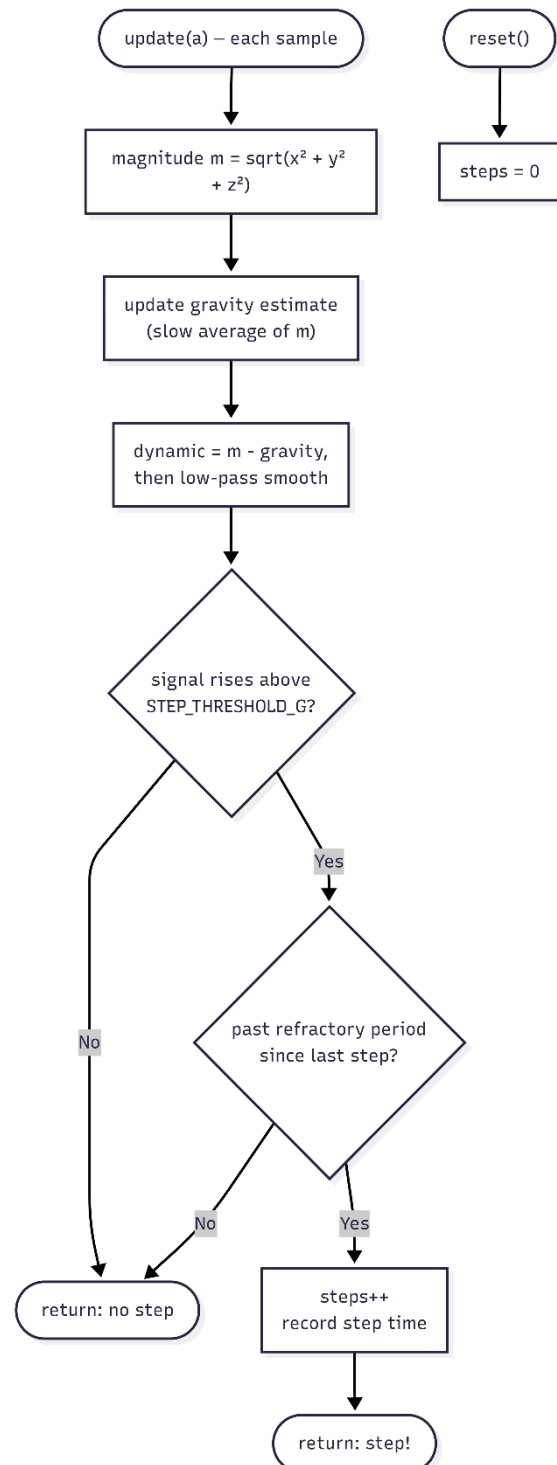


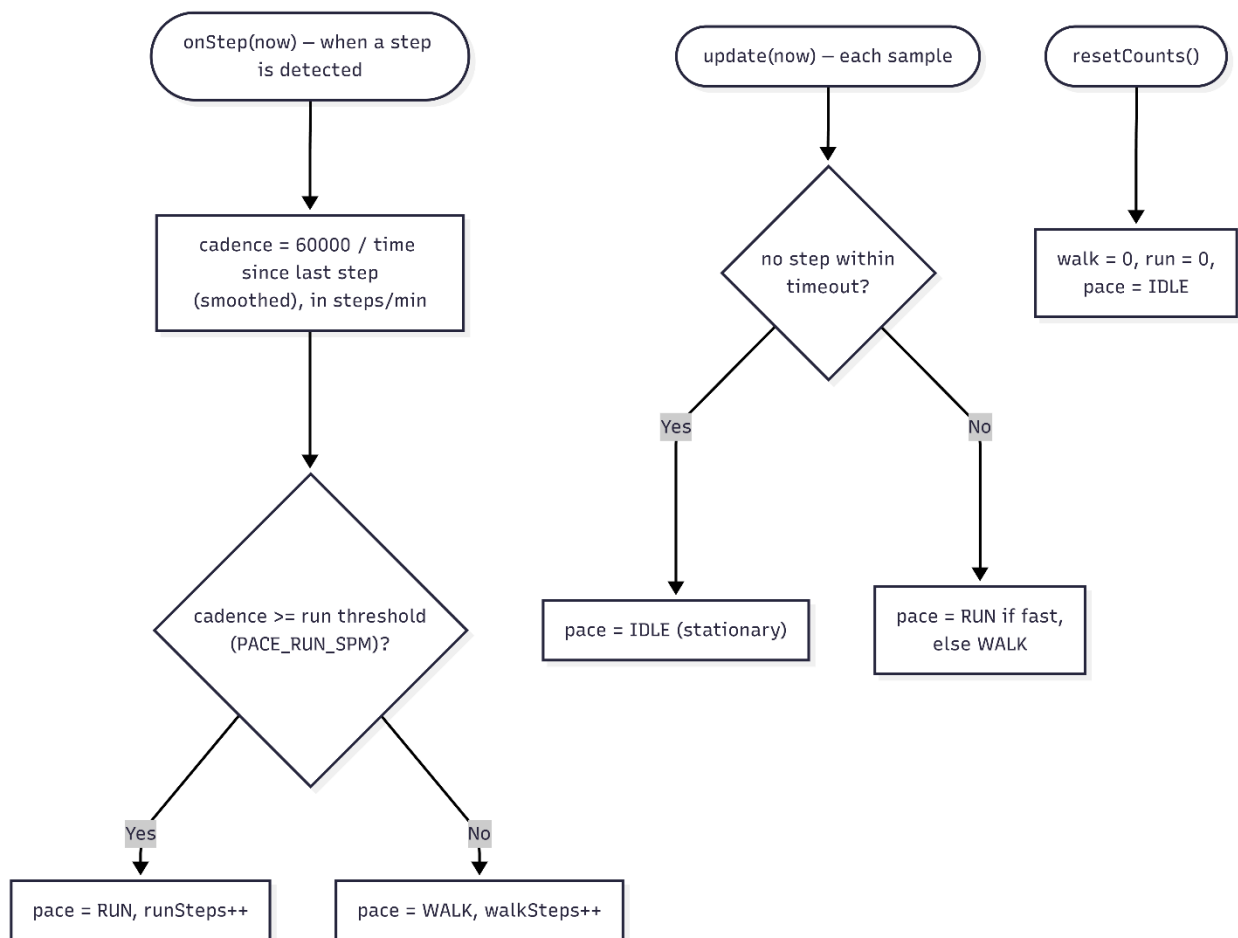
ADXL335 — Accelerometer Driver

- `begin()` — Configures the ADC ready to read the sensor's analog outputs.
- `useCalibration(cal)` — Stores the per-axis calibration (zero-g offset and sensitivity) used to convert readings.
- `readMv()` — Reads the raw X/Y/Z output voltages (in millivolts) from the analog pins.
- `read()` — Reads all three axes and converts each to acceleration in g using the active calibration; this is the main interface other modules use.

StepCounter — Step Detection

- `update(a)` — Processes one accelerometer sample: computes the vector magnitude, removes gravity, smooths the signal, and detects a step on a threshold crossing (gated by a refractory period). Returns true when a step is counted.
- `reset()` — Sets the step total back to zero.
- `getSteps()` / `lastDynamic()` — Accessors returning the current step count and the latest processed signal value (used for tuning).

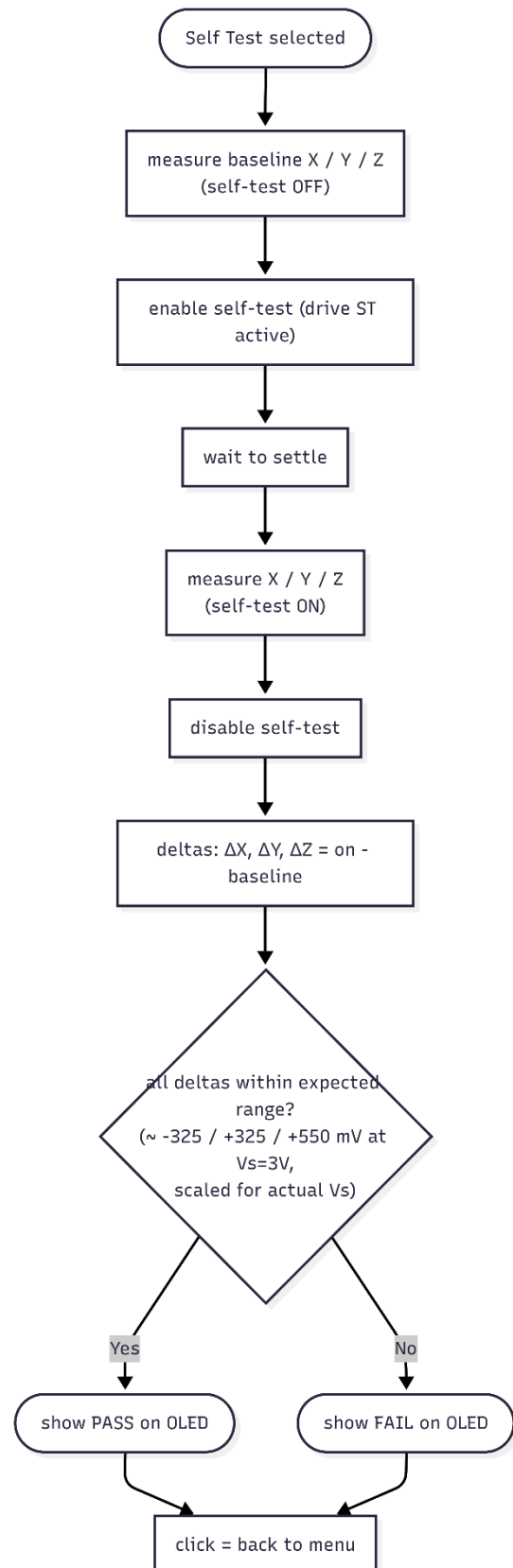




PaceIdentifier — Walking-Pace Classification

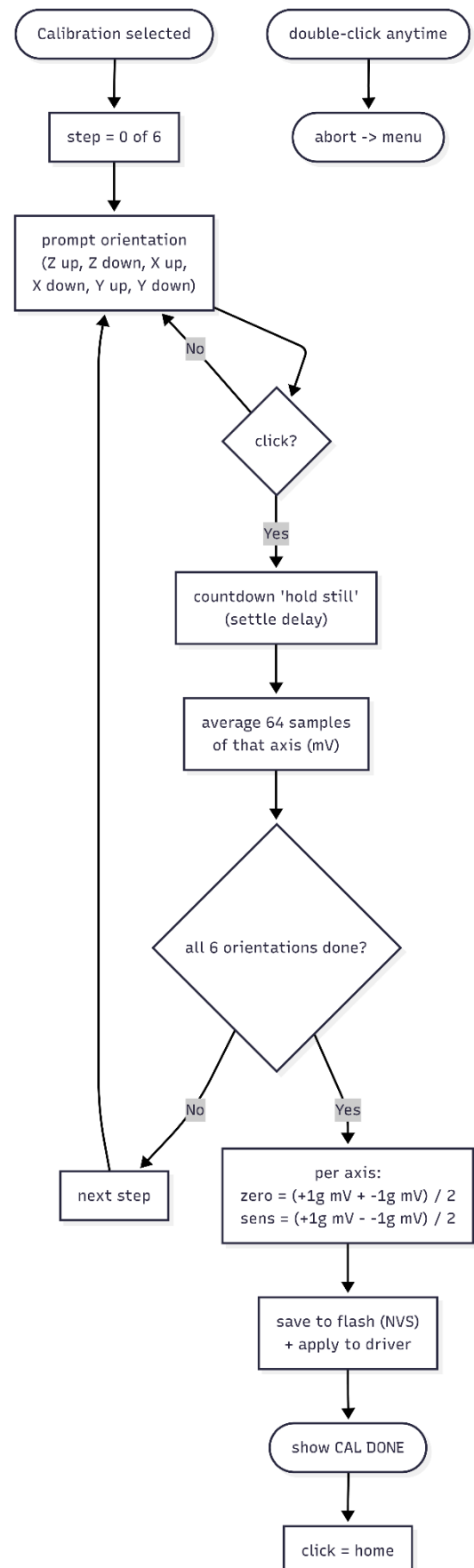
- **onStep(now)** — Called whenever a step is detected; estimates the cadence (steps/min) from the gap between steps and increments the walking or running tally accordingly.
- **update(now)** — Called every sample; sets the current pace to stationary if no recent step, otherwise walking or running based on cadence.
- **label()** — Returns the pace as text (IDLE / WALK / RUN) for the display.
- **resetCounts()** — Clears the per-pace step tallies and resets the pace state.

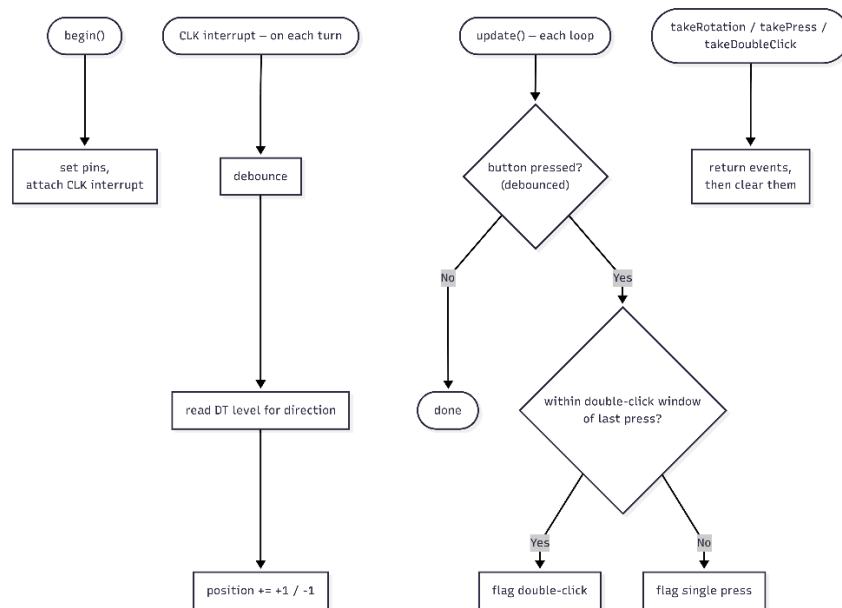
Self-Test Routine — Would record a baseline reading, electronically activate the sensor's built-in self-test, read the shifted output, and compute the change on each axis. It passes if all three axis shifts fall within the expected range from the datasheet, then reports PASS/FAIL on the screen.



Calibration — Calibration Storage & Math

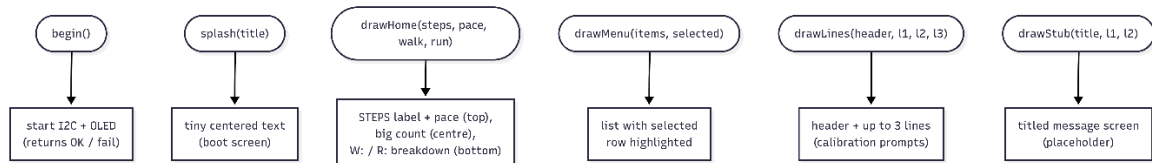
- `defaults()` — Returns the fallback calibration values used before any calibration has been run.
- `load()` — Reads the saved calibration from the ESP32's non-volatile flash storage (or defaults if none exists).
- `save(cal)` — Writes the calibration to flash so it persists across power cycles.





RotaryEncoder — Input Handling

- `begin()` — Configures the encoder pins and attaches the rotation interrupt.
- `clkIsr()` — Interrupt handler triggered on each rotation step; reads the direction line and increments/decrements a position counter.
- `update()` — Polled each loop to debounce the push-button and flag single presses and double-clicks.
- `takeRotation()` / `takePress()` / `takeDoubleClick()` — Hand the pending rotation/click events to the main program and clear them.
- `clearEvents()` — Discards any pending button events (used when switching screens).



Display — OLED Rendering

- `begin()` — Starts the I²C bus and the OLED; returns whether the screen responded.
- `splash(title)` — Draws the centered boot title screen.
- `drawHome(steps, pace, walk, run)` — Renders the home screen: step total, current pace, and walk/run breakdown.
- `drawMenu(items, selected)` — Renders the menu list with the selected row highlighted.
- `drawLines(header, ...)` — Renders a header plus up to three lines (used for calibration prompts).
- `drawStub(title, ...)` — Renders a simple titled message screen (used for the placeholder).

Embedded Software Testing and Validation

Embedded Software and Subroutine Testing & Validation

Threshold and Step Detection Subroutine

Purpose

To evaluate how accurately the system detects steps in any orientation, by determining the optimal threshold and timing settings that balance sensitivity against false step detection.

Testing Setup

Environment: A user wore/carried the step tracker and walked and ran under various path conditions, and the unit was also shaken in controlled bursts on a bench. A second person observed the live step count on the OLED and the acceleration data on a laptop.

Tools: The embedded step-tracker firmware running the step-detection algorithm, with the dynamic acceleration signal, threshold and step count streamed to a laptop via the PlatformIO Serial Monitor / Arduino Serial Plotter (115200 baud).

Inputs: Calibrated acceleration data (g) for the X, Y and Z axes from the ADXL335, reduced to a gravity-removed motion magnitude. Threshold (STEP_THRESHOLD_G) and minimum-step-interval (MIN_STEP_INTERVAL_MS) constants.

Tests Run

1. Determining the threshold (observing motion peaks against the noise floor).
2. Stationary (no false steps when still).
3. Walking in a circular path.
4. Walking in a square path with rounded corners.
5. Running in a circular path.
6. Running in a square path with rounded corners.
7. Counting in different orientations (flat, on-edge, vertical).
8. Reset steps.
9. Correct output on the OLED display.

Self-Test Subroutine

Purpose

To validate whether the accelerometer is functional, by monitoring the movement of the accelerometer beam when an electrostatic force is exerted on it as the self-test (ST) pin is set active.

Testing Setup

Environment: The step tracker was placed stationary on a flat surface. A second person connected the MCU to a laptop to record the change in acceleration readings (in mV) between the inactive and active self-test states.

Tools: The embedded firmware running the self-test routine, with per-axis acceleration readings (converted to mV) output to the laptop via the serial terminal.

Inputs: Acceleration data for the X, Y and Z axes from the ADXL335. The expected self-test output shift (datasheet: -325 mV X , $+325\text{ mV Y}$, $+550\text{ mV Z}$ at $V_s = 3\text{ V}$, scaled to the actual supply) and the allowed tolerance for the test to pass.

Tests Run

1. Stationary baseline reading (self-test off).
2. Self-test active — record the shifted readings.
3. Compare the per-axis shift (ΔX , ΔY , ΔZ) against the expected range (pass / fail).
4. Correct PASS/FAIL output on the OLED display.

Calibration Subroutine

Purpose

To confirm that the ADXL335 is producing the expected voltage under ± 1 g from gravity, to perform the known voltage-per-g conversion required for motion analysis, and to examine the static response of the sensor in all six orientations (+/- X, +/- Y, +/- Z).

Testing Setup

Environment: The step tracker was placed parallel to each of the 6 axes (+/- X, +/- Y, +/- Z) in turn.

Tools: The embedded firmware running the calibration routine, with the computed zero-g offset and voltage-per-g (sensitivity) for each axis output to the laptop via the serial terminal, and prompts/status shown on the OLED.

Inputs: Acceleration data (mV) for the X, Y and Z axes from the ADXL335. The allowed range of readings for calibration to pass.

Tests Run

1. Stationary (+/- X axis).
2. Stationary (+/- Y axis).
3. Stationary (+/- Z axis).
4. Calibration retained after a power cycle (loaded from flash on boot).
5. Correct output on the OLED display.

Pace Identification Subroutine

Purpose

To assess the ability to detect whether the movement of the step tracker is stationary, walking or running, and to keep a separate step count for each pace.

Testing Setup

Environment: A user wore the step tracker and walked and ran under various path conditions. A second person connected the MCU to a laptop to observe the change in step frequency and the pace classification, output via the serial terminal.

Tools: The embedded firmware taking the step count/timing as parameters and running the pace-identification algorithm, with the cadence and pace output via software and shown on the OLED.

Inputs: Step count and timing from the step-detection algorithm. The average walking frequency of humans and the cadence thresholds (PACE_RUN_SPM, stationary timeout).

Tests Run

1. Stationary.
2. Walking in circular motion.
3. Walking in a square path with rounded corners.
4. Running in circular motion.
5. Running in a square path with rounded corners.
6. Walking and running step counts sum to the total.
7. Correct output on the OLED display.

User Interface (Rotary Encoder & Menu) Subroutine

Purpose

To verify that the user can navigate the menu and operate every function reliably using the single rotary encoder.

Testing Setup

Environment: The step tracker was operated by hand while a second person observed the screen transitions on the OLED.

Tools: The embedded firmware running the encoder and menu state machine, with screen/state changes confirmed on the OLED display.

Inputs: Rotary encoder rotation (direction and detents) and button events (single press, double-click).

Tests Run

1. Double-click on the home screen opens the menu.
2. Rotation scrolls the menu one item per detent in the correct direction.
3. Single-click selects the highlighted item.
4. Each menu item (Step Counting, Self Test, Calibration, Reset) performs its action.
5. Correct output on the OLED display.

Test Report With All Specifications Recorded

Subroutine (unit) Test Results

ID	Subroutine	Specification	Expected result	Measured result	Pass/Fail
U1	ADXL335::read	Each axis responds to gravity over $\sim \pm 1$ g	All axes swing ≈ 600 mV; none stuck	X = 536 mV, Y = 654 mV, Z = 460 mV swing over tilt	Pass
U2	StepCounter::update	One step per motion peak; none when still	0 when still; +10 after 10 shakes	Stationary dyn ≤ 0.02 g, 0 steps; 10 shakes $\rightarrow 10$	Pass
U3	StepCounter refractory	No double-counting of one step	+1 per single step	1 sharp shake $\rightarrow$ count +1	Pass
U4	PaceIdentifier classify	IDLE still / WALK / RUN ≥ 140 spm	Correct label per state	Still $\rightarrow$ IDLE, walk $\rightarrow$ WALK, fast $\rightarrow$ RUN	Pass
U5	PaceIdentifier tallies	Walk + Run = total at all times	Sums match	W=42, R=8, total=50 $\rightarrow$ matches	Pass
U6	RotaryEncoder rotation	1 item per detent, correct direction	Highlight tracks rotation	CW down / CCW up, 1 item/detent	Pass
U7	RotaryEncoder button	Single = select, double = open menu	Both events correct	Double-click opened menu; single selected	Pass
U8	Calibration maths	Per-axis zero & sensitivity from ± 1 g	Zero $\approx$ mid-rail, sens $\approx 300\text{--}350$ mV/g	X: zero 1648 mV, sens 331 mV/g · Y: 1716, 328 · Z: 1809, 344	Pass
U9	Calibration persistence	Saved to flash, survives reboot	Boot reloads saved values	Boot log: "Calibration: loaded from flash"	Pass
U10	Display screens	Home, menu, calibration, splash render	All correct	All four screens rendered correctly	Pass

System Test Results (Against Design Requirements)

ID	Requirement	Specification	Expected result	Measured result	Pass/Fail
S1	Step counting (Req 3)	Count error $\leq 10\%$	50 ± 5 over 50 walked	Walked 50 $\rightarrow$ displayed 49 (2 % error)	Pass
S2	Orientation independence (Req 3)	Counts in any orientation	Each within $\pm 10\%$ of 20	Flat 20, on-edge 19, vertical 21	Pass
S3	Reset (Req 3)	Button resets counter	Total + W + R $\rightarrow 0$	All counts returned to 0	Pass
S4	Calibration (Req 2)	6-direction cal; success shown	CAL DONE; flat Z $\approx 1.0 \pm 0.1$ g	CAL DONE shown; flat Z = 1.01 g after cal	Pass
S5	Pace identification (Req 4)	Pace shown; per-pace counts kept	Correct transitions + tallies	IDLE $\rightarrow$ WALK $\rightarrow$ RUN correct; W/R updated	Pass
S6	Menu navigation	All items reachable/functional	Each performs its action	Step Counting, Calibration, Reset all worked	Pass
S7	Boot / stability	Boots to home; runs continuously	No resets over 5 min	Booted to home; stable for 5 min, no resets	Pass

Conclusion

All implemented subroutines and system-level requirements (step counting, calibration, pace identification, and the menu/UI) passed their specifications. Step counting met the $\leq 10\%$ accuracy target and remained reliable across orientations; calibration computed sensible per-axis values, persisted across power cycles, and improved the resting readings; pace identification classified correctly and maintained accurate per-pace tallies.